

Aufgabe 5 (Scala)

(13 Punkte)

Das **SnakeSort**-Verfahren sortiert eine zweidimensionale Matrix nach folgendem Verfahren: Zuerst werden im Schritt $i = 0$ die Werte zeilenweise abwechselnd auf- bzw. absteigend sortiert. Dann werden im Schritt $i = 1$ die Werte spaltenweise aufsteigend sortiert. Beide Schritte werden abwechselnd solange wiederholt, bis sich nach einem Schritt keine Änderung an der Matrix mehr ergibt.

Beispiel:

$$\begin{array}{ccccccc}
 \begin{bmatrix} 6 & 3 & 7 \\ 1 & 9 & 5 \\ 2 & 8 & 4 \end{bmatrix} & \xrightarrow[i=0]{\text{zeilenweise}} & \begin{bmatrix} 3 & 6 & 7 \\ 9 & 5 & 1 \\ 2 & 4 & 8 \end{bmatrix} & \xrightarrow[i=1]{\text{spaltenweise}} & \begin{bmatrix} 2 & 4 & 1 \\ 3 & 5 & 7 \\ 9 & 6 & 8 \end{bmatrix} & \xrightarrow[i=2]{\text{zeilenweise}} & \begin{bmatrix} 1 & 2 & 4 \\ 7 & 5 & 3 \\ 6 & 8 & 9 \end{bmatrix} \\
 m_0 & & m_1 & & m_2 & & m_3 \\
 \\
 \xrightarrow[i=3]{\text{spaltenweise}} & \begin{bmatrix} 1 & 2 & 3 \\ 6 & 5 & 4 \\ 7 & 8 & 9 \end{bmatrix} & \xrightarrow[i=4]{\text{zeilenweise}} & \begin{bmatrix} 1 & 2 & 3 \\ 6 & 5 & 4 \\ 7 & 8 & 9 \end{bmatrix} & \Rightarrow \text{fertig, da keine Änderung in Schritt } i = 4 \\
 & m_4 & & m_5
 \end{array}$$

Gegeben seien folgende Typdefinitionen und Funktionen:

```

type Row = List[Int]
type Matrix = List[Row]

```

```

def sortAsc: Row => Row // Sortiert Werte einer Zeile aufsteigend
def sortDesc: Row => Row // Sortiert Werte einer Zeile absteigend

```

Implementieren Sie Ihre Funktionen in *Scala* und ausschließlich unter Verwendung der Ihnen aus der Vorlesung und den Übungen bekannten Klassen und Funktionen. Die Verwendung von **var** und seiteneffektbehafteten Methoden/Klassen/Objekten ist untersagt.

- a) **sortCols(m)** sortiert die Werte der Matrix **m** spaltenweise in aufsteigender Reihenfolge, dabei spiegelt **transpose** die Matrix so, dass Zeilen zu Spalten und Spalten zu Zeilen werden. Geben Sie eine parallel arbeitende Variante **sortColsPar(m)** an.

```

def sortCols: Matrix => Matrix = m =>
  (for (col <- m.transpose) yield sortAsc(col)).transpose

```

```

def sortColsPar: Matrix => Matrix = m =>

```

- b) Die Funktion **sortRows(m)** sortiert die Werte der Matrix **m** zeilenweise abwechselnd auf- bzw. absteigend (beginnend mit *aufsteigend*) und gibt die sortierte Matrix zurück. Verwenden Sie zum Sortieren die gegebenen Funktionen **sortAsc** und **sortDesc**.

```
def sortRows: Matrix => Matrix =
```

- c) Die Funktion **sortHelper(i, m_i)** konstruiert einen unendlichen *Scala-Stream*, bestehend aus Matrix-Tupeln der Form (m_i, m_{i+1}) . Dabei ist m_i die Ausgangsmatrix für den Bearbeitungsschritt **i** (beginnend bei 0) und m_{i+1} dessen Ergebnismatrix.

Zum Sortieren können Sie auf die Funktionen **sortRows** und **sortCols** aus den vorhergehenden Teilaufgaben zurückgreifen.

```
def sortHelper: (Int, Matrix) => Stream[(Matrix, Matrix)] =
```

- d) Die Funktion **snakeSort(m)** sortiert die Matrix **m** nach dem beschriebenen Verfahren bis sich keine Änderung mehr ergibt und liefert die sortierte Matrix zurück. Verwenden Sie zur Implementierung die Funktion **sortHelper** aus der vorhergehenden Teilaufgabe.

Hinweise: Sie können davon ausgehen, dass die Startmatrix unsortiert ist. Matrizen dürfen hier mittels `==` und `!=` miteinander verglichen werden.

def snakeSort: Matrix $\Rightarrow$ Matrix =

Aufgabe 5 (Scala - Collatz)

(8 Punkte)

Implementieren Sie folgende Funktionen in *Scala* ausschließlich unter Verwendung der Ihnen aus der Vorlesung und den Übungen bekannten Klassen und Funktionen. Die Verwendung von **var** und von seiteneffektbehafteten Methoden/Klassen/Objekten ist untersagt.

- a) Die Collatz-Funktion $c(n)$ ist für alle natürlichen Zahlen $n > 0$ (muss nicht geprüft werden) definiert als:

$$c(n) = \begin{cases} \frac{n}{2} & \text{falls } n \text{ gerade ist} \\ 3n + 1 & \text{falls } n \text{ ungerade ist} \end{cases}$$

Sie soll nun durch folgende Hilfsfunktionen ausgedrückt werden:

```
def isEven: Int => Boolean = n => n % 2 == 0 // ist n gerade?
def cEven:  Int => Int      = n => n / 2
def cOdd:   Int => Int      = n => 3 * n + 1
```

```
def c: Int => Int = construct(isEven)(cEven, cOdd)
```

Definieren Sie die hierfür notwendige Funktion `construct(i)(e, o)` vollständig, d.h. inklusive der Funktionssignatur.

```
def construct:
```

- b) Eine Collatz-Folge $f(n)$ besteht aus den Elementen $f_i = c(f_{i-1})$ mit $i > 0$ und $f_0 = n$. Definieren Sie die Funktion `f(n)`, die die Collatz-Folge für die Zahl `n` in Form eines unendlichen Streams zurückgibt.

Beispiel: `f(5).take(6).toList == List(5, 16, 8, 4, 2, 1)`

```
def f: Int => Stream[Int] =
```

- c) Definieren Sie die Funktion `oddCount (n, k)`, die die Anzahl der ungeraden Zahlen in den ersten `k` Elementen der Collatz-Folge `f (n)` zurückliefert. Verwenden Sie hierzu unbedingt auch die Faltungsoperation `foldLeft`.

Beispiele:

```
oddCount (5, 3) == 1      mit f(5) = 5, 16, 8, ...
oddCount (12, 7) == 2     mit f(12) = 12, 6, 3, 10, 5, 16, 8, ...
oddCount (21, 5) == 1     mit f(21) = 21, 64, 32, 16, 8, ...
```

Hinweise:

- Sie können die Funktion `isEven` aus Teilaufgabe a) verwenden.
- **def** `foldLeft [B] (z: B) (f: (B, A) => B) : B`

```
def oddCount: (Int, Int) => Int =
```

Aufgabe 6 (Scala - Graphen färben)

(10 Punkte)

Implementieren Sie folgende Funktionen in *Scala* ausschließlich unter Verwendung der Ihnen aus der Vorlesung und den Übungen bekannten Klassen und Funktionen. Die Verwendung von **var** und von seiteneffektbehafteten Methoden/Klassen/Objekten ist untersagt.

In dieser Aufgabe sollen Sie einen Algorithmus zum Färben von ungerichteten Graphen mit möglichst wenigen Farben implementieren. Durch eine Kante verbundene Knoten müssen dabei unterschiedliche Farben erhalten. Jeder Knoten (Typ `Node`) besitzt eine eindeutige ID und eine Liste von IDs der Nachbarknoten.

Hinweise:

- In der Liste der Nachbarn kommen keine NodeIDs doppelt vor.
- Knoten besitzen keine Kanten auf sich selbst.

```
type NodeID = Int
type Color = Int
type Coloring = (NodeID, Color)
case class Node(id : NodeID, neighbors : List[NodeID])
```

- a) Implementieren Sie zunächst die Funktion `getFreeColor(cs)`. Die Funktion erhält eine Liste von Farben `cs` und soll die kleinste freie Farbe $c \geq 0$ zurückliefern, die nicht in der übergebenen Liste `cs` enthalten ist.

Hinweis: Sie dürfen die Funktion `contains` der Klasse `List` verwenden.

```
def contains(elem: Any): Boolean
```

Beispiele:

```
getFreeColor(List(0,1,7,1,3,7)) == 2
getFreeColor(List(47,11))      == 0
```

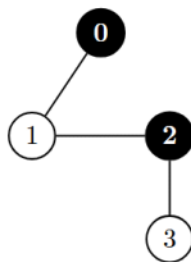
```
def getFreeColor: List[Color] => Color =
```

- b) Implementieren Sie die rekursive Funktion `colorGraph(ns)(cs)`, die eine Liste ungefärbter Knoten `ns` und eine Liste bereits ermittelter Färbungen (Colorings) `cs` erhält. Die Funktion ermittelt für alle Knoten in `ns` eine Färbung und liefert die Gesamtliste (bestehend aus `cs` und den neuen Färbungen) zurück. Verwenden Sie zur Implementierung die vorgegebene Funktion `nextFreeColor(cs)(n)`, die für einen Knoten `n` die kleinste, in der Nachbarschaft noch nicht verwendete Färbung ermittelt.

```
def nextFreeColor: List[Coloring] => Node => Coloring = ...
```

```
def colorGraph: List[Node] => List[Coloring] => List[Coloring] =
```

Hilfestellung



Knoten:

```
val n0 = Node(0, List(1))
val n1 = Node(1, List(0,2))
val n2 = Node(2, List(1,3))
val n3 = Node(3, List(2))
```

Graph:

```
val ns = List(n0, n1, n2, n3)
```

```
colorGraph(ns)(Nil):
```

```
List((3,1), (2,0), (1,1), (0,0))
```

```
≡ List((3, weiß), (2, schwarz), (1, weiß), (0, schwarz))
```

Aufgabe 4 (Scala – Teilersummen)

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Die Teilersumme $fs(n)$ der natürlichen Zahl n sei die Summe ihrer ganzzahligen Teiler ohne n .

Beispiele:

n	4	5	6	7	8
$fs(n)$	$1 + 2 = 3$	$1 = 1$	$1 + 2 + 3 = 6$	$1 = 1$	$1 + 2 + 4 = 7$

- a) Die Funktion $fs(n)$ ($n > 0$ wird garantiert) soll wie folgt ausgedrückt werden:
- ```
def facs: Int => List[Int] // facs(n) liefert die Teiler von n (ohne n)
def sum: List[Int] => Int // sum(xs) summiert alle Zahlen in xs
def fs: Int => Int = construct(facs)(sum)
```

Geben Sie die hierfür notwendige Funktion `construct(f)(s)` inklusive Signatur an:

```
def construct:
```

- b) Die Folge  $fss(n_0)$  bestehe aus den Tupeln  $(n_i, fs(n_i))$  mit  $i \geq 0$  und  $n_{i+1} = n_i + 1$ . Definieren Sie die Funktion  $fss(n_0)$ , die die obige Folge als unendlichen Stream zurückgibt.

**Beispiel:** `fss(4).take(5).toList == List((4,3), (5,1), (6,6), (7,1), (8,7))`

```
def fss: Int => Stream[(Int, Int)] =
```

- c) Geben Sie die Funktion `count(n0, m)` an, die aus den ersten  $m$  Elementen von  $fss(n_0)$  die Anzahl der Tupel  $(n_i, fs(n_i))$  ermittelt, für die  $n_i > fs(n_i)$  gilt.

Verwenden Sie unbedingt `foldLeft[B](z: B)(f: (B, A) => B): B` aus `List[A]`.

**Beispiel:** `count(4, 5) == 4` mit  $fss(4) = (4,3), (5,1), (6,6), (7,1), (8,7), \dots$

```
def count: (Int, Int) => Int =
```

**Aufgabe 6 (Scala – Ausgehende Kanten)**

(8 Punkte)

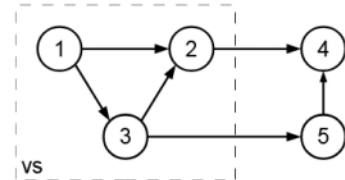
Verwenden Sie *ausschließlich* aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und *seiteneffektbehaftete Methoden/Klassen/Objekte* strikt untersagt.

Gegeben seien gerichtete Graphen  $G$  folgender Form:

```
type V = Int // Knoten
type E = (V, V) // Kante: (Startknoten, Endknoten)
type G = List[E]
```

**Beispiel:**

```
val graph = List((1, 2), (1, 3), (3, 2), (2, 4), (3, 5), (5, 4))
```



- a) Definieren Sie die Funktion `outEdges(vs, g)`, die parallel diejenigen Kanten in  $g$  filtert und zurückgibt, die direkt von Knoten der Knotenliste  $vs$  zu Knoten außerhalb von  $vs$  führen. Sie dürfen die Methode `contains(t: T): Boolean` von `List[T]` verwenden.

**Beispiel** (siehe Bild): `outEdges(List(1, 2, 3), graph) == List((2, 4), (3, 5))`

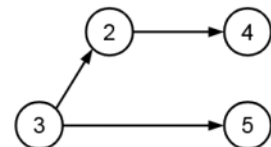
```
def outEdges: (List[V], G) => List[E] = (vs, g) =>
```

- b) Ergänzen Sie die Funktion `tree(v, g)`, die einen beliebigen gerichteten Baum (als Kantenliste) mit Wurzel  $v$  bestimmt, welcher alle von  $v$  aus in  $g$  erreichbaren Knoten enthält. `helper(vs)` bearbeitet rekursiv (mittels `outEdges`) die bereits besuchten Knoten  $vs$ .

**Beispiel** (auch andere gerichtete Bäume möglich!):

```
tree(3, graph) == List((3, 2), (2, 4), (3, 5))
```

```
def tree: (V, G) => G = (v, g) => {
 def helper: List[V] => G = vs =>
```



```
 helper(List(v))
 }
```

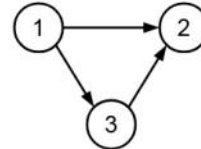
**Aufgabe 5 (Scala – Gerichteter Graph)**

(7 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Ein gerichteter Graph  $G$  ist als Liste von Adjazenzen  $A(v, \text{out})$  gegeben.  $v$  ist dabei ein Knoten der Form  $V(id)$ .  $\text{out}$  ist die Liste der direkt von  $v$  erreichbaren Nachbarknoten:

```
case class V(id: Int)
case class A(v: V, out: List[V])
type G = List[A]
```



**Beispiel:**

```
val v1 = V(1); val v2 = V(2); val v3 = V(3);
val graph: G = List(A(v1, List(v2, v3)), A(v2, Nil), A(v3, List(v2)))
```

- a) `isIn(v, vs)` prüft, ob der Knoten  $v$  in der Knotenliste  $vs$  enthalten ist. Geben Sie eine parallel arbeitende Variante `isInPar(v, vs)` an.
- ```
def isIn: (V, List[V]) => Boolean = (v, vs) => vs.filter(_ == v).nonEmpty
def isInPar: (V, List[V]) => Boolean = (v, vs) =>
```

- b) Geben Sie die Funktion `find(v, g)` an, die alle Knoten in g zurückgibt, von denen v direkt erreichbar ist. Sie können dabei die Funktion `isIn` verwenden.

Beispiele: `find(v2, Nil) == List()`; `find(v2, graph) == List(V(1), V(3))`

```
def find: (V, G) => List[V] =
```

Aufgabe 6 (Scala – Lauflängencodierung)

(11 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Bei der folgenden Lauflängencodierung wird ein mehrfach aufeinander folgendes Zeichen *c* als Tupel (c, k) codiert, wobei *k* die Anzahl der Wiederholungen angibt.

Beispiel: "aaabbccc..." wird zu $(\text{'a'}, 3), (\text{'b'}, 2), (\text{'c'}, 3), \dots$

- a) Definieren Sie die Funktion `encodeHead(cs)`, die aus einem Zeichenstrom *cs* mittels Bedarfsauswertung das erste codierte Tupel *t* ermittelt und dieses zusammen mit dem Reststrom *rs* als Tupel (t, rs) zurückgibt. Nehmen Sie an, dass *cs* nicht leer ist.

Beispiel: `encodeHead("aaabbccc".toStream) == ((a,3),Stream(b, ?))`

Tipps: Verwenden Sie die Methoden `takeWhile`, `dropWhile` und `lengthInStream`.

```
def encodeHead: Stream[Char] => ((Char, Int), Stream[Char]) = cs => {
    val h = cs.head
```

```
}
```

- b) Definieren Sie die Funktion `encode(cs)`, die aus einem Zeichenstrom *cs* mittels Bedarfsauswertung und `encodeHead(cs)` einen Strom codierter Tupel erstellt. **Beispiel:**

```
encode("aaabbccc".toStream).take(3).toList == List((a,3), (b,2), (c,3))
```

Tipp: Verwenden Sie die Methode `isEmpty` in `Stream`.

```
def encode: Stream[Char] => Stream[(Char, Int)] =
```

- c) Definieren Sie die Funktion `decode(ts)`, die eine Liste codierter Tupel *ts* decodiert. Verwenden Sie unbedingt `foldRight[B](z: B)(f: (A, B) => B): B` aus `List[A]`.

Beispiel: `decode(List(('a',3), ('b',2), ('c',3))) == List(a,a,a,b,b,c,c,c)`

Tipp: `List.fill(k)(c)` erstellt eine Liste bestehend aus *k* mal dem Zeichen *c*.

```
def decode: List[(Char, Int)] => List[Char] =
```

Aufgabe 5 (Scala – Listenoperationen)

(9 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und *seiteneffektbehaftete Methoden/Klassen/Objekte* strikt untersagt.

- a) Geben Sie die **rekursive** Funktion `flatten(ts)` an, die eine Liste `ts` von Tupeln der Form `(Char, Char)` in eine Liste der Zeichen aus `ts` umwandelt. Dabei sollen Reihenfolge und Anzahl der Zeichen entsprechend ihrem Auftreten in `ts` erhalten bleiben.

Beispiel: `scala> flatten(List(('a', 'b'), ('a', 'c')))`
 (REPL) `res0: List[Char] = List(a, b, a, c)`

def `flatten: List[(Char, Char)] => List[Char] =`

- b) Geben Sie die **parallel** arbeitende Version `countPar(cs)` von `count(cs)` an. Diese ermittelt zu jedem Zeichen `c` in `cs` ein Tupel `(c, n)`, wobei `n` angibt, wie oft `c` in `cs` vorkommt.

Beispiel: `scala> count(List('a', 'b', 'a', 'c'))`
 (REPL) `res1: List[(Char, Int)] = List((a,2), (b,1), (a,2), (c,1))`

def `count: List[Char] => List[(Char, Int)] = cs =>`
 for (`c <- cs`) **yield** (`c, cs.filter(_ == c).size`)
def `countPar: List[Char] => List[(Char, Int)] = cs =>`

- c) Geben Sie die Funktion `distinct(xs)` an, die aus der Liste `xs` alle Duplikate entfernt. Die Reihenfolge der Elemente in der Ergebnisliste ist irrelevant.

Verwenden Sie **unbedingt** `foldLeft[B](z: B)(f: (B, A) => B): B` aus `List[A]`.

Beispiel: `scala> distinct(List(('a',2), ('b',1), ('a',2), ('c',1)))`
 (REPL) `res2: List[(Char, Int)] = List((c,1), (b,1), (a,2))`

Tipp: Sie dürfen `contains(elem: Any): Boolean` aus der Klasse `List` verwenden.

def `distinct: List[(Char, Int)] => List[(Char, Int)] =`

Aufgabe 6 (Scala – Lauflängendekodierung)

(9 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Bei der folgenden Lauflängendekodierung wird ein Strom von Tupeln (c, k) zu einem Strom von Zeichen dekodiert. Pro Tupel wird dabei das Zeichen c jeweils k -mal wiederholt.

Beispiel: `Stream(('a', 3), ('b', 4), ('c', 2))` wird dekodiert zu
`Stream('a', 'a', 'a', 'b', 'b', 'b', 'b', 'c', 'c')`

- a) Definieren Sie die Funktion `decodeTuple(t)`, die ein einzelnes Tupel $t = (c, k)$ zu einem Strom von k Zeichen c dekodiert. Dies muss mittels einer Bedarfsauswertung erfolgen, die jeweils nur einzelne Zeichen verarbeitet.

Beispiel: Folgende Anweisung wertet nur das erste von 4711 Zeichen aus:

(REPL) `scala> decodeTuple(('a', 4711))`
`res0: Stream[Char] = Stream(a, ?)`

def `decodeTuple: ((Char, Int)) => Stream[Char] =`

- b) Definieren Sie die Funktion `decode(ts)`, die einen Strom ts von Tupeln unter Verwendung von `decodeTuple` und mittels Bedarfsauswertung dekodiert.

Beispiel: Folgende Zeile dekodiert nur 4 der 7 Zeichen:

(REPL) `scala> decode(Stream(('a', 3), ('b', 4))).take(4).toList`
`res1: List[Char] = List(a, a, a, b)`

Tipps: Zwei Streams können durch `#:::` mittels Bedarfsauswertung verbunden werden. Die Verwendung von `.head` und `.tail` der Klasse `Stream` erleichtert die Bearbeitung.

def `decode: Stream[(Char, Int)] => Stream[Char] =`

Aufgabe 5 (Scala – Game of Life)

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

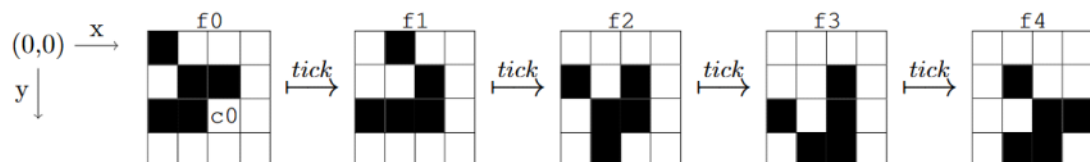
Beim *Game of Life* sind Zellen auf einem begrenzten rechteckigen Feld so angeordnet, dass an jeder Position (x, y) des Feldes immer genau ein `Cell`-Objekt ist:

```
case class Cell(x: Int, y: Int, alive: Boolean)
type Field = List[Cell]
```

Zellen im Inneren des Feldes haben jeweils 8 Nachbarn, Zellen am Rand haben weniger. Alle Zellen ändern ihren Zustand gleichzeitig per `tick` nach folgenden Regeln:

- Eine Zelle mit $n < 2$ oder $n > 3$ lebenden Nachbarn (`alive`) ist nach dem `tick` tot.
- Eine Zelle mit $n = 3$ lebenden Nachbarn ist nach dem `tick` lebendig.
- In allen anderen Fällen behält eine Zelle ihren Zustand bei.

Beispiel: Eine Zelle ist `alive` (■) oder `!alive` (□).



- a) Definieren Sie die Funktion `aliveNeighbors` **inklusive** Signatur, welche die Anzahl n der lebenden Nachbarn einer Zelle ermittelt. Verwenden Sie dazu eine vorgegebene und übergebene Funktion die Nachbarn einer Zelle `c` im Feld `f` liefert.

```
def neighbors: Field => Cell => List[Cell] = ... // ist vorgegeben
val n: Int = aliveNeighbors(neighbors(f0))(c0) // n == 3

def aliveNeighbors:
```

- b) Definieren Sie die Funktion `tick(f)` gemäß o.g. Regeln. Verwenden Sie `aliveNeighbors`:

```
def tick: Field => Field =
```

- c) Definieren Sie die Funktion `game(f)`, die den Strom aller Felder ab `f` (inkl.) berechnet:

```
def game: Field => Stream[Field] =
```

Aufgabe 6 (Scala – Missionare und Kannibalen)

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

m Missionare und k Kannibalen wollen einen Fluss mit einem Boot überqueren, dabei dürfen auf jeder Uferseite nie mehr Kannibalen als Missionare sein. Es sind folgende Typen vorgegeben:

State Zustand mit der Anzahl an Personen pro Uferseite sowie der Bootsstellung.

Transition Gültiger Zustandsübergang (Bootsfahrt).

type Solution = List[Transition]

- a) Definieren Sie die Funktion `first(sols)`, die aus `sols` die erste (kleinster Index) nicht-leere Solution zurückgibt. Gibt es keine derartige Solution, so wird `Nil` zurückgegeben.

Verwenden Sie unbedingt `foldLeft[B](z: B)(f: (B, A) => B): B` aus `List[A]`.

Beispiele:

```
val sol1: Solution = ... // eine nicht-leere Solution
val sol2: Solution = ... // eine nicht-leere Solution
first(List(Nil, sol1, Nil, sol2)) == sol1
first(List(Nil, Nil, Nil)) == Nil
```

```
def first: List[Solution] => Solution =
```

- b) Die Funktion `solve(start, end)` ermittelt eine `Solution`, die von `start` zu `end` führt. Ergänzen Sie die **rekursive** Funktion `helper(zs, sol)`, die die Liste der besuchten Zustände `zs` und die zugehörigen Zustandsübergänge `sol` verwaltet.

Zur Erleichterung ist die Funktion `passage(st)` vorgegeben, die ausgehend vom Zustand `st` nur die gültigen direkten Folgezustände und den jeweils dabei verwendeten Zustandsübergang ermittelt:

```
def passage: State => List[(State, Transition)] = ... // ist vorgegeben
```

Es müssen folgende Fälle berücksichtigt werden:

- Ist der Endzustand `end` erreicht, soll `sol` zurückgegeben werden.
- Wurde ein Zustand doppelt besucht, soll `Nil` zurückgegeben werden.
- Gibt es keine Lösung, soll `Nil` zurückgegeben werden, andernfalls eine beliebige.

Tipp: Denken Sie an die Funktion `first` aus Teilaufgabe a).

```
def solve: (State, State) => Solution = (start, end) => {
  // helper wird am Ende von solve aufgerufen
  def helper: (List[State], Solution) => Solution = {

    case (s::ss, sol) if s == end =>

    case (s::ss, sol) if ss.contains(s) =>

    case (s::ss, sol) =>

  }

  helper(List(start), Nil) // Ergebnis von solve
}
```

Aufgabe 5 (Scala: Sortiervverfahren)

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und *seiteneffektbehaftete Methoden/Klassen/Objekte* strikt untersagt.

In dieser Aufgabe setzen Sie schrittweise ein einfaches Sortiervverfahren in Scala um. Verwenden Sie dazu **keine** Sortierfunktionen aus der Java- oder Scala-API.

- a) Implementieren Sie die Funktion `insert(ls, v)`, die die Zahl `v` so in die aufsteigend sortierte Liste `ls` einfügt, dass die Ergebnisliste ebenfalls aufsteigend sortiert ist.

```
def insert: (List[Int], Int) => List[Int] =
```

- b) Implementieren Sie die Funktion `sort(ls)`, die die Liste `ls` aufsteigend sortiert. Verwenden Sie **unbedingt** `foldRight[B](z: B)(f: (A, B) => B): B`, um die übergebene Liste abzuarbeiten und mittels `insert` eine sortierte Liste aufzubauen.

```
def sort: List[Int] => List[Int] =
```

- c) Definieren Sie die Funktion `sortAllWith(f) (ls)` (inklusive Signatur), die eine Sortierfunktion `f` und eine Liste von Listen `ls` erhält. `sortAllWith` soll jede in `ls` enthaltene Liste von `Int`-Werten mit der Funktion `f` sortieren. Verwenden Sie für Ihre Lösung unbedingt eine Listenabstraktion („list comprehension“).

Beispiel:

```
sortAllWith(sort) (List (List (4, 3), List (2, 1, 3)))  
== List (List (3, 4), List (1, 2, 3))
```

```
def sortAllWith:
```

Aufgabe 6 (Scala: Monte Carlo)

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

In dieser Aufgabe implementieren Sie ein Monte Carlo-Verfahren zur Bestimmung der Schnittfläche zweier Ellipsen. Gegeben sind dabei folgende Definitionen:

```
// Datentyp fuer Ellipsen
type Ellipse = ...

// isInside(e, x, y) prueft, ob der Punkt (x, y) in der Ellipse e liegt
def isInside: (Ellipse, Double, Double) => Boolean = ...

// randomStream gibt einen unendlichen Stream von Zufallszahlen zurueck
def randomStream: Stream[Double] = ...
```

- a) Definieren Sie die Funktion `testInside` (inklusive Signatur), so dass bei einem Aufruf von `testInside(e1, e2) (x, y)` genau dann `true` zurückgegeben wird, wenn der Punkt `(x, y)` sowohl in der Ellipse `e1` als auch in `e2` enthalten ist.

```
def testInside:
```

- b) Definieren Sie die Funktion `tupleStream(s)`, die einen unendlichen Stream `s` aus Double-Zahlen (z.B. `randomStream`) in einen Stream aus Zahlentupel (Punkte) überführt, indem je zwei aufeinander folgende Zahlen zu einem Tupel zusammengefasst werden.

Beispiel:

```
tupleStream(randomStream).take(2).toList
== List((1.0, 2.0), (3.0, 4.0))
```

```
def tupleStream: Stream[Double] => Stream[(Double, Double)] =
```

- c) Vervollständigen Sie die Funktion `monteCarlo(e1, e2)(count)`. Diese Funktion soll `count` zufällige Punkte auswählen (benutzen Sie dazu `pointStream`). Anschließend soll parallel die Anzahl der Punkte bestimmt werden, die sowohl in `e1` als auch in `e2` liegen.

```
def monteCarlo: (Ellipse, Ellipse) => Int => Int = (e1, e2) => count => {  
    // Stream aus Punkten  
    val pointStream = tupleStream(randomStream)
```

```
}
```

Aufgabe 5 (Huffman-Codierung)

(18 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Hinweis zur Scala-API der Klasse `List` (sowie `Stream` entsprechend):

```
def count(p: (A) => Boolean): Int
    Counts the number of elements in the traversable or iterator which satisfy a predicate.

def filter(p: (A) => Boolean): List[A]
    Selects all elements of this traversable collection which satisfy a predicate.
```

Gegeben seien folgende case-Klassen für binäre Bäume:

```
abstract class Node(val freq: Int)
case class Leaf(f: Int, c: Char) extends Node(f)
case class Inner(f: Int, zero: Node, one: Node) extends Node(f)
```

- a) Die Funktion `freq` soll für jedes *disjunkte* Zeichen c der Eingabeliste l die Anzahl f der Vorkommen von c in l bestimmen:

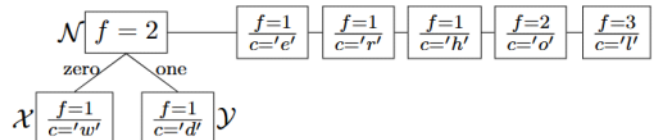
Beispiel: Ergebnis für
`freq(List(h, e, l, l, o, w, o, r, l, d))` $\rightsquigarrow$ $\begin{bmatrix} f=1 \\ c=h' \end{bmatrix} \begin{bmatrix} f=1 \\ c=e' \end{bmatrix} \begin{bmatrix} f=3 \\ c=l' \end{bmatrix} \begin{bmatrix} f=2 \\ c=o' \end{bmatrix} \begin{bmatrix} f=1 \\ c=w' \end{bmatrix} \begin{bmatrix} f=1 \\ c=r' \end{bmatrix} \begin{bmatrix} f=1 \\ c=d' \end{bmatrix}$

```
def freq: List[Char] => List[Leaf] = {
```

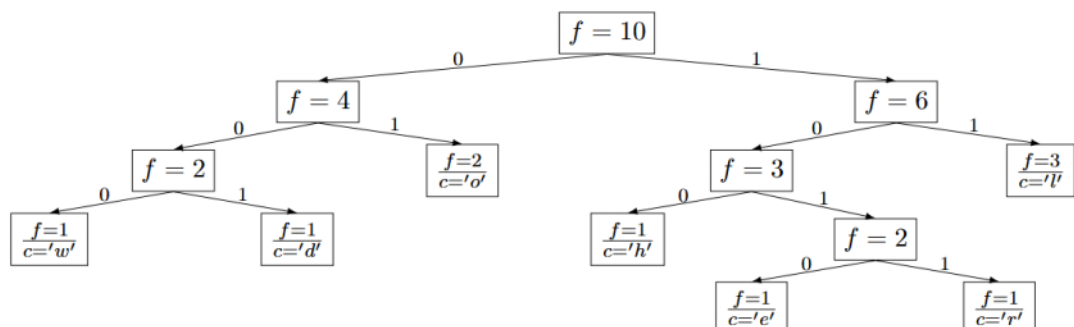
```
}
```

- b) Die Funktion `merge` sortiert zuerst die Liste l so, dass die bzgl. f ($\mapsto freq$ in `Node`) *kleinsten* Knoten $\mathcal{X}$ und $\mathcal{Y}$ am Anfang der Liste stehen (bereits vorgegeben). Dann verschmilzt sie $\mathcal{X}$ und $\mathcal{Y}$ zu einem neuen inneren Knoten $\mathcal{N}$, wobei $\mathcal{X}$ zum `zero`-Kind, $\mathcal{Y}$ zum `one`-Kind und das neue f von $\mathcal{N}$ die Summe der f s der Unterknoten von $\mathcal{N}$ wird. Anschließend werden $\mathcal{X}$ und $\mathcal{Y}$ aus der Liste entfernt und $\mathcal{N}$ hinzugefügt.

Beispiel:
 Erster Durchlauf von
`merge(freq(l))` für $l =$
`List(h, e, l, l, o, w, o, r, l, d)` $\rightsquigarrow$



`merge` wiederholt den gesamten Vorgang, bis nur noch ein Knoten (die Wurzel des Baums) in der Liste übrig bleibt. **Beispiel:** `merge(freq(List(h, e, l, l, o, w, o, r, l, d)))`:



```
def merge: List[Node] => Node = l => {
  def mergeHelper: List[Node] => Node = {
    case Nil => ??? // undefined
```

```
  }
  mergeHelper(l.sortBy(_.freq))
}
```

- c) Der Pfad von der Wurzel des resultierenden Baums zu einem Blatt mit Zeichen c stellt die Huffman-Codierung von c dar:

$$b \rightsquigarrow \overbrace{100}^h \overbrace{1010}^e \overbrace{11}^l \overbrace{11}^l \overbrace{01}^o \overbrace{000}^w \overbrace{01}^o \overbrace{1011}^r \overbrace{11}^l \overbrace{001}^d \dots$$

Die Funktion `getEnc` ermittelt die Huffman-Codierung jedes Zeichens im Baum als Tupel aus Zeichen und Bit-Folge und gibt alle Tupel als Liste zurück. Verwenden Sie unbedingt je eine `for`-comprehension über die Codierungen der Unterbäume!

```
type CharEnc = (Char, List[Int])

def getEnc: Node => List[CharEnc] = {
  case Leaf(f, c) => List((c, Nil))
  case Inner(f, zero, one) =>
```

```
    (for (e <- getEnc(zero))
```

```
    }
```

- d) Die Funktion `encode` überführt einen (eventuell unendlichen) **Strom** von Zeichen in die Bit-Darstellung ihrer jeweiligen Huffman-Codierung (siehe Beispiel bei Teilaufgabe c)).
Hinweis: Verwenden Sie `map`, `filter` und `flatten` geeignet!

```
def encode: List[CharEnc] => Stream[Char] => Stream[Int] = enc =>
```

Aufgabe 5 (Scala: Soziales Netzwerk)

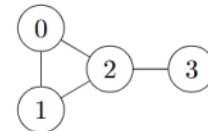
(7 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

In einem sozialen Netzwerk sind Personen durch Zahlen und Freundschaftsbeziehungen durch einen Graphen als Adjazenzliste dargestellt:

```
type Graph = List[List[Int]]
```

```
val exg: Graph = List(List(1, 2), List(0, 2),  
                        List(0, 1, 3), List(2))
```



- a) Implementieren Sie die Funktion `containsAll(ps, ts)`, die genau dann `true` zurückgibt, wenn die Liste der Personen `ps` mindestens alle Personen aus `ts` enthält. Benutzen Sie unbedingt `foldLeft[B](z: B)(op: (B, A) => B): B`. Beispiel:

```
containsAll(List(0, 1, 2, 3), List(1, 3)) == true  
containsAll(List(0, 2, 3), List(1, 3)) == false
```

```
def containsAll: (List[Int], List[Int]) => Boolean = (ps, ts) =>
```

- b) Implementieren Sie die Funktion `allFriends(g, vs)`, die eine Liste aller gemeinsamen Freunde der Personen aus `vs` zurückgibt. Beispiel:

```
allFriends(exg, List(0, 3)) == List(2)
allFriends(exg, List(2, 3)) == Nil
```

Hinweis: Eine Person ist nie mit sich selbst befreundet. Verwenden Sie zur Lösung die Funktion `containsAll` und eine Listenabstraktion (for-comprehension).

```
def allFriends: (Graph, List[Int]) => List[Int] = (g, vs) =>
```

Aufgabe 6 (Scala: Newton-Verfahren)

(13 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Das Newton-Verfahren sucht eine Nullstelle einer Funktion. Gegeben seien folgende Definitionen:

```
// Beispielfunktion:
val f0: Double => Double = x => x*x - x - 2

// Berechnet die Ableitung einer Funktion f
def deriv: (Double => Double) => Double => Double = f => ...

// Prüft, ob zwei Zahlen fast gleich sind
def equal: (Double, Double) => Boolean = ...
```

- a) Für eine Funktion f und deren Ableitung f' ist das Newton-Verfahren definiert über

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Implementieren Sie die Funktion `newton` (inklusive Signatur). Diese liefert den *unendlichen* Strom aller Iterationen x_i des Newton-Verfahrens beginnend bei `x0` für die übergebene Funktion und deren Ableitung. Damit ist `x0` das erste Element des resultierenden Stroms. Ihre Signatur muss zu folgendem Aufruf passen:

```
val x0: Double = 1 // Startwert
val s0: Stream[Double] = newton(f0, deriv(f0))(x0)

def newton:
```

- b) Möglicherweise konvergiert das Newton-Verfahren nicht, d.h. es wird keine Nullstelle gefunden. Dies kann durch den Scala-Datentyp `Option` ausgedrückt werden:

```
abstract class Option[+T]  
case object None extends Option[Nothing]  
case class Some[+T](v: T) extends Option[T]
```

Ergänzen Sie `getZero(m)(s)`. Befindet sich unter den ersten `m` Gliedern des *unendlichen* Stroms `s` ein Paar aufeinanderfolgender Zahlen, das fast gleich (`equal`) ist, soll eine dieser Zahlen zurückgegeben werden. Andernfalls liefert die Funktion `None`.

```
def getZero: Int => Stream[Double] => Option[Double] = m => s =>
```

Aufgabe 2 (Scala: Verschlüsselung)

(8 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und *seiteneffektbehaftete Methoden/Klassen/Objekte* strikt untersagt.

Beim nachfolgend beschriebenen Verschlüsselungsverfahren wird ein Buchstabe *c* mit einem Passwortzeichen *k* verschlüsselt, indem in Zeile *c* und Spalte *k* einer Verschlüsselungsmatrix das Ausgabezeichen nachgeschlagen wird. Rechts ist eine Beispielmatrix dargestellt. Damit ergibt sich beispielhaft:

```
Nachricht: A A B D ...
Passwort:  B C C B ...
Ergebnis: N O P Q ...
```

	A	B	C	...
A	M	N	O	...
B	N	O	P	...
C	O	P	Q	...
D	P	Q	R	...
...		...		

Gegeben sei eine in folgender Hilfsfunktion definierte Verschlüsselungsmatrix:

```
// Verschlüsselungsmatrix; c: Buchstabe; k: Passwortzeichen
def encMatrix: Char => Char => Char = c => k => // ...
```

- a) Vervollständigen Sie die Funktion `cycle(key)`, die aus einer Liste von Zeichen `key` einen unendlichen Strom erzeugt, indem der Inhalt der Liste unendlich oft wiederholt wird.

Beispiel: `cycle("KEY".toList).take(10).mkString == "KEYKEYKEYK"`

```
def cycle: List[Char] => Stream[Char] = key =>
```

- b) Implementieren Sie die Funktion `cipher` (*inklusive* Signatur), die mit einer Verschlüsselungsmatrix `f` und einem Schlüssel `key` eine (möglicherweise unendlich lange) Nachricht `msg` verschlüsselt. Bringen Sie das Passwort durch Wiederholung auf eine ausreichende Länge. Ihre Funktion muss zu folgendem Beispielaufwurf passen:

```
val msg: Stream[Char] = "MESSAGE".toStream
val key: List[Char] = "KEY".toList
val r: Stream[Char] = cipher(encMatrix)(key)(msg)
```

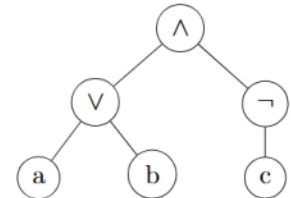
```
def cipher:
```

Aufgabe 6 (Scala: Kantorowitsch-Bäume)

(12 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Kantorowitsch-Bäume dienen zur symbolischen Darstellung von Termen. Innere Knoten repräsentieren Operatoren, deren Operanden jeweils in den Unterbäumen stehen, während Blätter Variablen sind. Das nebenstehende Beispiel zeigt den Term $(a \vee b) \wedge \neg c$ (Infix-Notation). Gegeben seien folgende Datentypen für Variablennamen bzw. für Baumknoten sowie die beiden Hilfsfunktionen:



```
type Name = Char
```

```
abstract class KTree
```

```
case class And(l: KTree, r: KTree) extends KTree
```

```
case class Or(l: KTree, r: KTree) extends KTree
```

```
case class Not(c: KTree) extends KTree
```

```
case class Var(n: Name) extends KTree
```

```
// wertet KTree kt mit Variablenbelegung va aus
```

```
def eval: KTree => List[(Name, Boolean)] => Boolean = kt => va => kt match {
  // ...
}
```

```
// liefert Liste aller Variablen im KTree
```

```
def vars: KTree => List[Name] = { // ...
}
```

- a) Ergänzen Sie die Funktion `all` so, dass sie für *alle* Variablen aus der Eingabeliste den Strom aller möglichen Belegungen generiert. Verwenden Sie unbedingt eine `for-comprehension` mit **zwei** Generatoren, jedoch keine `Stream`- oder `List`-Konkatenation (`::` und `#::` sind erlaubt):

Beispiel:

```
all(List('a', 'b')).toList == List(List(('a', false), ('b', false)),
  List(('a', false), ('b', true)), List(('a', true), ('b', false)),
  List(('a', true), ('b', true)))
```

```
def all: List[Name] => Stream[List[(Name, Boolean)]] = {
  case Nil => Stream(Nil)
  case v :: vs =>
```

```
}
```

- b) Ergänzen Sie die folgende Funktion so, dass sie für den übergebenen *Kantorowitsch-Baum* den Strom aller möglichen Variablenbelegungen generiert, für die der Baum zu `true` ausgewertet werden würde – verwenden Sie dazu *unbedingt* `filter`:

```
def sat: KTree => Stream[List[(Name, Boolean)]] = {
```

```
}
```

- c) Ergänzen Sie schließlich die folgende Funktion, die die übergebene Liste von Bäumen mit dem And-Operator verknüpft und zu einem Baum zusammensetzt. Verwenden Sie *unbedingt* `foldLeft` statt Rekursion:

```
def and: List[KTree] => KTree = {
```

```
  case ls => ??? // undefiniert
```

```
}
```

Aufgabe 2 (Scala: Binäre Suchbäume)

(9 Punkte)

Verwenden Sie *ausschließlich* aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und *seiteneffektbehaftete Methoden/Klassen/Objekte* strikt untersagt.

Gegeben sei folgende generische Darstellung eines binären Suchbaums:

```
abstract class Node[+E]  
case object Leaf extends Node[Nothing]  
case class Inner[E](elem: E, l: Node[E], r: Node[E]) extends Node[E]
```

Die einzufügenden Elemente sollen mittels einer Vergleichsfunktion angeordnet werden, die hier beispielhaft für `String` definiert ist:

```
def strCmp: (String, String) => Int = (a, b) => a.compareTo(b)  
// Erg. negativ/0/positiv, falls a kleiner/gleich/groesser b
```

- a) Vervollständigen Sie die generische Funktion `insert[E]` (**inklusive** Signatur), die mittels einer Vergleichsfunktion das gegebene Element in den übergebenen Baum einfügt, so dass weiterhin kleinere Elemente im Kind `l` und größere im Kind `r` stehen. Duplikate sollen **nicht** erneut eingefügt werden. Ihre Signatur muss zu folgendem Beispielaufruf passen:
- ```
insert[String](strCmp)("foo")(Leaf) == Inner("foo", Leaf, Leaf)
```

```
def insert[E]:
```

- b) Ergänzen Sie die Funktion `build(ls)`, die aus einer `String`-Liste einen binären Suchbaum erzeugt. Nutzen Sie **unbedingt** `foldLeft[B](z: B)(op: (B, A) => B): B` und `insert` aus Teilaufgabe a) mit der Vergleichsfunktion `strCmp`.

```
def build: List[String] => Node[String] =
```

**Aufgabe 6 (Scala: Bergsteigeralgorithmus)**

(11 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Ausgehend von einem Lösungskandidaten untersucht der Bergsteigeralgorithmus alle benachbarten Lösungen und wählt den besten Nachbarn als neuen Lösungskandidaten. Wird keine bessere Lösung gefunden, wurde das Optimum erreicht. Folgende Klasse modelliert generisch den Lösungsraum:

```
abstract class Solution {
 def quality: Double // Qualitaet der Loesung
 def neighbors: List[Solution] // Benachbarte Loesungen
}
```

- a) Realisieren Sie die Funktion `findBest(s)`, die alle Nachbarn des Lösungskandidaten `s` untersucht und diejenige Lösung `s2` mit der höchsten `quality` ermittelt. Ist `quality` von `s2` höher als von `s`, wird `s2` zurückgegeben, ansonsten `s`.

```
def findBest: Solution => Solution = s =>
```

- b) Implementieren Sie die Funktion `hillClimbing(s)`, die einen unendlichen Strom der jeweils besten Lösung für jeden Schritt des Verfahrens liefern soll, angefangen mit `s`. Damit ist das erste Element `s`, das zweite Element ist `s2 = findBest(s)` usw.

```
def hillClimbing: Solution => Stream[Solution] = s =>
```

- c) Ergänzen Sie `optimum(sls)`, so dass die Funktion den unendlichen Strom `sls` auf zwei aufeinanderfolgende, gleiche Lösungen durchsucht. Dieses Optimum soll die Funktion zurückgeben.

```
def optimum: Stream[Solution] => Solution = sls =>
```

- d) Nun ist die Hilfsfunktion `swap` gegeben, die zwei Elemente einer Liste vertauscht:

```
// Vertauscht Elemente der Liste ls an den Indizes i und j. (i, j >= 0)
def swap[E]: (List[E], Int, Int) => List[E] = (ls, i, j) => ...
```

Vervollständigen Sie die Funktion `swapNeighbors`, die alle Nachbarn einer `Int`-Liste generiert, die durch systematisches Vertauschen zweier (nicht zwingend benachbarter) Elemente entstehen. Verwenden Sie unbedingt die vorgedruckte Listenabstraktion.

**Beispiel:** `swapNeighbors(List(1,2,3)) == List(List(2,1,3), List(3,2,1), List(1,3,2))`

```
def swapNeighbors: List[Int] => List[List[Int]] = ps =>
 for (
```

```
) yield swap(ps,
```

**Aufgabe 2 (Scala: Umgekehrte Polnische Notation)**

(9 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Die *Umgekehrte Polnische Notation* (UPN) ist eine Schreibweise für arithmetische Ausdrücke, bei der die Operatoren *hinter* den Operanden stehen. Somit ist der Ausdruck `38 3 1 + +` äquivalent zur Infix-Notation `38 + (3 + 1)`. Gegeben sind folgende Definitionen:

```
abstract class Elem
case class Num(n: Int) extends Elem
case object Add extends Elem // Addition (einzige Op. in dieser Aufgabe)
```

```
def parseDecimal: String => Int = // ...
def parseRomanNum: String => Int = // ...
```

- a) Implementieren Sie die Funktion `asElems` inklusive Signatur, die mit einer als gegeben anzunehmenden Konvertierungsfunktion (z.B. `parseDecimal`) eine Liste von Zeichenketten `ls` in eine äquivalente Liste von `Elem`-Subklassen überführt. Nehmen Sie an, dass `ls` nur gültige Zeichenketten enthält. Ihre Definition muss zu folgenden Beispielen passen:

```
asElems(parseDecimal)(List("38", "3", "1", "+", "+"))
 == List(Num(38), Num(3), Num(1), Add, Add)
asElems(parseRomanNum)(List("XX", "I", "+")) == List(Num(20), Num(1), Add)
```

```
def asElems:
```

- b) Vervollständigen Sie die Funktion `eval(es)`, die einen gültigen UPN-Ausdruck als Liste von `Elem`-Subklassen erhält und diesen zu einem `Int`-Ergebnis auswertet. Verwenden Sie zur Lösung unbedingt `foldLeft[B](z: B)(op: (B, A) => B): B`.

Beispiel: `eval(List(Num(38), Num(3), Num(1), Add, Add)) == 42`

Die Auswertung erfolgt von links nach rechts: Zahlen werden auf einen Stapel (`List`) gelegt; bei einem `Add` werden die zwei obersten Zahlen durch ihre Summe ersetzt.

```
def eval: List[Elem] => Int =
 _.foldLeft[List[Int]](List())((rs, e) => e match {
```

**Aufgabe 5 (Scala: Fibonacci-Codierung)**

(11 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Jede positive ganze Zahl  $z$  kann als Summe verschiedener Fibonacci-Zahlen geschrieben werden. Welche Fibonacci-Zahl in dieser Summendarstellung vorkommt, kann durch ein gesetztes bzw. nicht gesetztes Bit in einer Bitfolge (der *Fibonacci-Codierung* von  $z$ ) angezeigt werden.

Diese Aufgabe behandelt die Decodierung eines Fibonacci-codierten Worts. Dabei werden die Positionen  $i$  der gesetzten Bits (`true`) bestimmt. Zur Decodierung werden für die gesetzten Positionen  $i$  die Werte von  $\text{fib}(i + 2)$  aufsummiert. Diese Summe ergibt die Fibonacci-codierte Zahl  $z$ . Wenn z. B. in einem Code-Wort die Bits 0 und 3 gesetzt sind, werden  $\text{fib}(0 + 2) = 1$  und  $\text{fib}(3 + 2) = 5$  zum Ergebnis 6 addiert:

|                     |                   |                    |                    |                   |                                                                          |
|---------------------|-------------------|--------------------|--------------------|-------------------|--------------------------------------------------------------------------|
| Codierung von $z=6$ | <code>true</code> | <code>false</code> | <code>false</code> | <code>true</code> |                                                                          |
| Position $i$        | <b>0</b>          | 1                  | 2                  | <b>3</b>          | $\rightsquigarrow \text{fib}(0 + 2) + \text{fib}(3 + 2) = 1 + 5 = 6 = z$ |
| $\text{fib}(i + 2)$ | <b>1</b>          | 2                  | 3                  | <b>5</b>          |                                                                          |

Die Bits eines Worts sind in dieser Aufgabe als `Boolean` dargestellt. Gegeben ist außerdem der unendliche Strom aller Fibonacci-Zahlen (0, 1, 1, 2, 3, 5, ...):

```
val fibStream: Stream[Int] = // ...
```

- a) Setzen Sie die Funktion `fibDecode(cs)` um, die ein Fibonacci-codiertes Wort `cs` erhält und dieses in die zugehörige positive ganze Zahl umwandelt. Beispiel:

```
fibDecode(Stream(true, false, false, true)) == 6
```

```
def fibDecode: Stream[Boolean] => Int =
```

- b) Entwickeln Sie die Funktion `splitCode(bs)`, die einen unendlichen Strom von Bits erhält und jedes darin vorkommende Code-Wort zu einem eigenen Strom zusammenfasst. Als Trennzeichen werden zwei aufeinanderfolgende, gesetzte Bits definiert, von denen das zweite Bit (im Beispiel unterstrichen) *verworfen* werden soll. Gehen Sie daher davon aus, dass `true, true` nie Teil eines Code-Worts ist. Beispiel:

```
splitCode(Stream(true, false, false, true, true, true, true, ...))
 == Stream(Stream(true, false, false, true),
 Stream(true), ...)
```

Verwenden Sie zur Lösung unbedingt Mustervergleich auf Streams.

```
def splitCode: Stream[Boolean] => Stream[Stream[Boolean]] = {
```

```
}
```

- c) Realisieren Sie die Funktion `decodeStream(n, bs)`, die parallel die ersten `n` ganzen Zahlen berechnet, die im Bitstrom `bs` codiert sind. Beispiel:

```
decodeStream(2, Stream(true, false, false, true, true, true, true, ...))
 == Stream(6, 1)
```

*Tipp:* Verwenden Sie `splitCode` und `fibDecode` aus den vorherigen Teilaufgaben. Es genügt eine parallele Ausführung der `fibDecode`-Aufrufe.

```
def decodeStream: (Int, Stream[Boolean]) => Stream[Int] = (n, bs) =>
```

**Aufgabe 4 (Scala: Versteckte Botschaften)**

(9 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

In vielen, ausreichend langen Texten kann man „versteckte Botschaften“ finden, indem man alle Leer- und Satzzeichen entfernt und ab einer bestimmten Startposition beginnend immer das  $n$ -te Zeichen betrachtet.

Im Prinzip geht es ums Finden eines Startpunkts und einer Schrittweite, in diesem Fall ist der Startpunkt 2 und die Schrittweite 16.

- a) Vervollständigen Sie die Funktion `prepare(cs)`, die aus dem Strom `cs` alle Zeichen entfernt, die keine Buchstaben sind (`isLetter`), sowie alle verbleibenden Buchstaben in Kleinbuchstaben (`toLowerCase`) umwandeln soll.

```
def prepare: Stream[Char] => Stream[Char] =
 .(.isLetter)
 .(_.toLowerCase)
```

Nehmen Sie **ab jetzt** an, dass alle Char-Ströme bereits mit `prepare` verarbeitet wurden.

- b) Ergänzen Sie die Funktion `getStarts(p, cs)`, die einen Strom aller Positionen (`Int`) liefern soll, an denen das Zeichen `p` in `cs` vorkommt.

```
def getStarts: (Char, Stream[Char]) => Stream[Int] = (p, cs) =>
 cs. (Stream.from(0)) // Strom der Indizes
 .
 .map()
```

- c) Die Funktion `nth(cs, n)` soll einen Strom der Zeichen in `cs` zurückgeben, die sich an den Positionen  $0, n, 2 \cdot n, 3 \cdot n, \dots$  in `cs` befinden. Füllen Sie die Lücken geeignet. Ihre Lösung soll sowohl endliche als auch unendliche `cs` unterstützen.

```
def nth: (Stream[Char], Int) => Stream[Char] = (cs, n) =>
 if (cs.isEmpty)
 else
 nth()
```

- d) Vervollständigen Sie die Funktion `find(ps, cs, m)`, die alle möglichen Startpositionen `s` und Schrittweiten `i` ( $1 \leq i \leq m$ ) ermitteln soll, für die man die (nicht-leere) „versteckte Botschaft“ `ps` in `cs` finden kann. Sie können davon ausgehen, dass `cs` bereits mit `prepare` verarbeitet wurde.

**Tipp:** Verwenden Sie die Stream-Funktion `startsWith` sowie die Funktionen aus den Teilaufgaben b) und c).

```
def find: (List[Char], Stream[Char], Int) => Stream[(Int, Int)]
 = (ps, cs, m) =>
```

```
 for (
)

) yield
```

**Aufgabe 5 (Scala: Halde)**

(11 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Neben der binären Halde gibt es noch weitere Haldenvarianten zum wiederholten Finden des kleinsten Elements einer Menge. Eine solche ist durch folgende Datentypdefinitionen gegeben:

```
abstract class Heap
case object Empty extends Heap
case class Tree(min: Int, cs: List[Tree]) extends Heap // MinHeap
```

Dabei darf das Element *min* nie größer sein als ein Element in den Unterbäumen *cs*. Das Einfügen in eine Halde kann auf das Verschmelzen zweier Halden zurückgeführt werden:

```
def insert: (Int, Heap) => Heap = (e, h) => merge(Tree(e, List()), h)
```

- a) Implementieren Sie die Funktion `merge(h1, h2)`, die zwei Halden *h1* und *h2* mit paarweise verschiedenen Elementen zu einer Halde verschmilzt. Dabei wird diejenige Halde *h*, die **nicht** das neue kleinste Element enthält, als *neues erstes* Kind der anderen Halde eingefügt, sofern *h* nicht-leer ist.

```
def merge: (Heap, Heap) => Heap = {
```

```
}
```

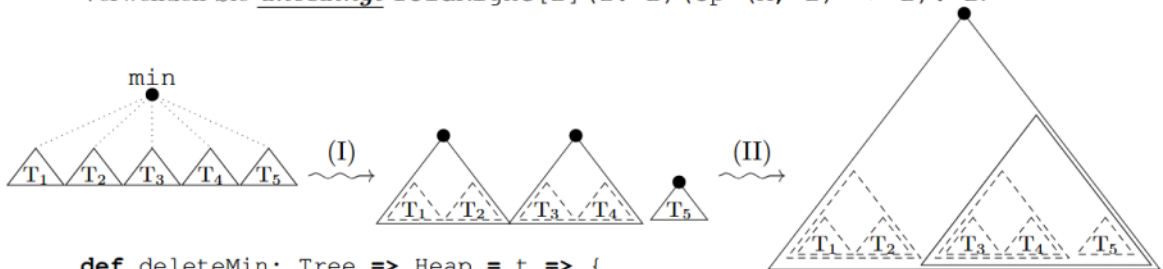
- b) Vervollständigen Sie die Hilfsfunktion `combWith(f) (ls)` inklusive Signatur, die auf jeweils zwei benachbarte Elemente der Liste `ls` die Funktion `f` anwendet. `f` soll jedes Listenelement nur einmal erhalten, somit ist `combWith(f) (List (A, B, C, D)) == List (f(A, B), f(C, D))`. Verwenden Sie für Listen ungerader Länge für den letzten Aufruf `Empty` als zweites Argument. Ihre Umsetzung muss zu folgendem Beispiel passen:

```
val ls: List[Heap] = List(Tree(3,Nil), Tree(4,Nil), Tree(5,Nil))
val rs: List[Heap] = combWith(merge) (ls)
rs == List(merge(ls(0), ls(1)), merge(ls(2), Empty))
```

```
def combWith:
```

- c) Ergänzen Sie die Funktion `deleteMin(t)`, die das kleinste Element aus der Halde `t` entfernt, indem sie die direkten Kinder von `t` zu einer Halde umformt: Verschmelzen Sie (I) zunächst benachbarte Kinder mit `merge` und Teilaufgabe b), bevor Sie (II) von hinten nach vorne alle Elemente der Liste nacheinander mit `merge` verschmelzen. Dieses Vorgehen vermeidet degenerierte Bäume.

Verwenden Sie unbedingt `foldRight[B] (z: B) (op (A, B) => B) : B`.



```
def deleteMin: Tree => Heap = t => {
```

```
}
```

**Aufgabe 5 (Scala: Mengenüberdeckung)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Beim Mengenüberdeckungsproblem gibt es eine Grundmenge  $U$  (z. B. Produkte) und eine Sammlung  $\mathcal{S}$  von Teilmengen mit Kosten (z. B. angebotene Produktpakete mit Preis). Gesucht ist eine Auswahl von Teilmengen aus  $\mathcal{S}$  mit minimalen Gesamtkosten, deren Vereinigungsmenge alle Elemente aus  $U$  umfasst (z. B. günstigste Einkaufsmöglichkeit für alle Produkte).

In dieser Aufgabe wird die Grundmenge  $U$  vereinfachend als `List[String]` dargestellt. Ferner sind folgende Definitionen gegeben:

```
abstract class SetColl
case object Empty extends SetColl
case class Sets(es: List[String], cost: Int, next: SetColl) extends SetColl
// Sets ist eine verkettete Liste von Mengen mit Kosten
```

- a) Ergänzen Sie die Funktion `costSum(s)`, die die Gesamtkosten der Lösung `s` berechnet und zurückgibt.

**Beispiel:**

```
val s = Sets(List("Milch", "Mehl"), 13, Sets(List("Eier"), 3, Empty))
costSum(s) == 16
```

```
def costSum: SetColl => Int = {
```

```
}
```

- b) Implementieren Sie die Funktion `allOf(us, s)`, die überprüft, ob alle Elemente aus `us` in `s` enthalten sind.

**Beispiel:**

```
allOf(List("Mehl", "Eier", "Milch"), s) == true
```

```
def allOf: (List[String], SetColl) => Boolean = {
```

```
}
```

Nun ist zusätzlich folgende Definition gegeben:

```
def better: (SetColl, SetColl) => SetColl = ...
```

Die Funktion `better(s1, s2)` prüft, ob `s1` oder `s2` `Empty` ist. Falls ja, wird die jeweils andere Lösung zurückgegeben. Falls nicht, wird die Lösung mit niedrigerer Kostensumme geliefert.

- c) Vervollständigen Sie die Funktion `setCover(s, us, r)`, die eine Sammlung von Teilmengen `s`, eine Liste aller verfügbaren Elemente `us` und eine (partielle) Lösung `r` erhält. Ist `s` leer, soll die Funktion `r` zurückgeben, sofern dies eine gültige Lösung ist (andernfalls `Empty`). Enthält `s` noch mindestens eine Teilmenge `e`, soll die Funktion rekursiv untersuchen, ob eine Lösung mit oder ohne `e` kostengünstiger ist (Tipp: Verwenden Sie `better`). Sie können annehmen, dass beim ersten Aufruf von `setCover` die (partielle) Lösung `Empty` ist.

```
def setCover: (SetColl, List[String], SetColl) => SetColl = {
```

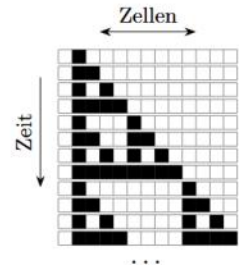
```
}
```

**Aufgabe 6 (Scala: Regel 60)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

„Regel 60“ ist ein einfacher zellulärer Automat. Dieser besteht aus einem unendlichen, eindimensionalen Band aus Zellen, die entweder aktiv (■) oder inaktiv (□) sein können. Dieses Band verändert sich schrittweise, wobei in jedem Schritt für jede Zelle der eigene Zustand sowie der des linken Nachbarn vor dem Schritt betrachtet wird. Unterscheiden sich die Zustände dieser beiden Zellen (z. B. ■ und □), wird die Zelle ■, andernfalls □. Somit *beeinflusst* der Zustand einer Zelle an Position  $x$  im nächsten Schritt die Zellen an Position  $x$  und  $x+1$ . Die zeitliche Entwicklung ausgehend von einer aktiven Zelle ist rechts beispielhaft gezeigt.



Das unendliche Band wird dargestellt durch einen Strom, der in aufsteigender Reihenfolge die **Indizes** der aktiven Zellen enthält. Halten Sie sich bei Ihren Antworten an den Vordruck.

- a) Ergänzen Sie die Funktion `next(s)`, die ausgehend von einem Zustand `s` den Nachfolgezustand generiert. Dazu werden für jede aktive Zelle mit Index  $x$  die beeinflussten Zellen  $x$  und  $x + 1$  betrachtet. Wird eine Zelle genau einmal beeinflusst, ist sie im nächsten Schritt aktiv, bei keiner oder zweimaliger Beeinflussung hingegen inaktiv.

```
def next: Stream[BigInt] => Stream[BigInt] =
 // Generiere beeinflusste Zellen:
 - [] (x => Stream(x, x + 1))
 // Entferne Zellen, die zweimal beeinflusst werden:
 .foldRight(Stream.empty[BigInt])((e, res) =>
 if ([])
 []
 else
 []
)
```

- b) Vervollständigen Sie die Funktion `steps(f)(is)` (*inklusive* Signatur), die die zeitliche Entwicklung des zellulären Automaten darstellen soll. Dazu erhält sie eine Zustandsübergangsfunktion `f` sowie einen Ausgangszustand `is`. Das Ergebnis soll dann der unendliche Strom `is, f(is), f(f(is)), ...` sein.

**Beispiel:**

```
val s: Stream[Stream[BigInt]] = steps(next)(Stream(1)).take(3)
s.map(_.toList).toList == List(List(1), List(1, 2), List(1, 3))
```

```
def steps:
```

**Aufgabe 5 (Scala: Stromzerlegung)**

(8 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Bei der Verarbeitung von Dokumenten werden oftmals zunächst die darin enthaltenen bedeutungstragenden Elemente, z. B. Namen oder Zahlen, bestimmt. Beispiel:

```

 PFP Klausur2020 Scala
 Name Name Zahl Name

```

Zur Identifizierung der Zeichen, die Bestandteile von Namen bzw. Zahlen sind, und zum Erkennen der zu überspringenden Leerzeichen stehen folgende Funktionen zur Verfügung:

```

def isWhitespace: Char => Boolean = ... // Leerzeichen
def isLetter: Char => Boolean = ... // Buchstaben
def isDigit: Char => Boolean = ... // Ziffern

```

- a) Ergänzen Sie die Funktion `tokenizeHead(f)(cs)` (*inklusive* Signatur), welche erst alle Leerzeichen am Anfang von `cs` überspringt. Dann trennt `tokenizeHead` das längste Präfix des resultierenden Stroms ab, sodass die Funktion `f` für jedes Zeichen `true` liefert, und gibt ein Tupel aus diesem Präfix als Liste und dem restlichen Strom ohne Präfix zurück.

Ihre Definition *muss* zu folgenden Beispielen passen:

```

tokenizeHead(isLetter)("_PFP_Klausur".toStream)
 == (List('P', 'F', 'P'), "_Klausur".toStream)

tokenizeHead(isDigit)("4_7_11".toStream)
 == (List('4'), "_7_11".toStream)

tokenizeHead(isLetter)("42Scala".toStream)
 == (List(), "42Scala".toStream)

```

```

def tokenizeHead:

{
 val skippedWS =

}

```

- b) Vervollständigen Sie die Funktion `names(cs)`, die alle Namen aus dem möglicherweise unendlichen Strom `cs` extrahiert. Sie können annehmen, dass `cs` nur (aus Buchstaben bestehende) Namen und Leerzeichen enthält. **Tipp:** Nutzen Sie `tokenizeHead`.

```

def names: Stream[Char] => Stream[List[Char]] = {
 case Stream.Empty => Stream.Empty
 case cs =>

}

```

**Aufgabe 6 (Scala: Längste Rundreisen)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Ermitteln Sie für ein gegebenes Netz (Graph) von Verbindungen zwischen Ausflugszielen eine Rundreise, die eine maximale Anzahl von Zielen besucht (längster Zyklus).

- a) Vervollständigen Sie die Funktion `maxLength(css)`, die aus einer Liste `css` von Zyklen (gegeben als Liste der besuchten Knotennummern) den längsten Zyklus liefert.

```
def maxLength: List[List[Int]] => List[Int] = {
 case Nil => ??? // undefined -> Exception
 case h::ts => ts.foldRight(h) (
 (e, r) => if (e.length > r.length) e else r
)
}
```

Graphen sind in dieser Aufgabe durch eine Liste dargestellt, die für jeden Knoten die Liste der Nachfolger enthält (Adjazenzliste `gs`; Knoten werden über den Index ihres Eintrags identifiziert).

- b) Ergänzen Sie die Funktion `makeLongest(gs, rs)`, die den übergebenen, nicht-leeren Pfad `rs` zu einem längsten Zyklus erweitert (gibt es keinen Zyklus, soll `Nil` zurückgegeben werden). Dazu soll sie alle Nachbarn `n` des Endknotens (`rs.last`) von `rs` betrachten:
- Ist `n` der erste Knoten des Pfades `rs`, wurde ein Zyklus gefunden.
  - Ist `n` ein anderer Knoten des Pfades, kann mit `n` kein Zyklus gebildet werden, der den ganzen Pfad enthält.
  - Andernfalls wird `rs` um `n` erweitert und die Suche mit dem neuen Pfad fortgesetzt.

Von den so bestimmten längsten Zyklen aller Nachbarn soll der längste ausgewählt werden.

```
def makeLongest: (List[List[Int]], List[Int]) => List[Int] =
 (gs, rs) => maxLength(gs(rs.last).map(n =>
 if (n == rs.head)
 rs ::: List(n)
 else if (rs.contains(n))
 Nil
 else
 makeLongest(gs, rs:::List(n))
))
```

- c) Setzen Sie in der Funktion `longest` die Suche nach einem längsten Zyklus um, indem Sie für jeden möglichen Startknoten  $0 \leq n < gs.length$  die Funktion `makeLongest` aufrufen und das passende Ergebnis auswählen.

```
def longest: List[List[Int]] => List[Int] = gs =>
```

```
maxLength(for (x <- List.range(0,gs.length)) yield makeLongest(gs, List(x)))
```

**Aufgabe 2 (Scala: Ohne Punkt und Komma)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

*Scriptio continua* ist eine Schreibweise, bei der Wörter ohne Leer- oder Satzzeichen hintereinander geschrieben werden. Entwickeln Sie in dieser Aufgabe ein Verfahren, um die einzelnen Worte aus einem solchen Text zu extrahieren. Dazu soll gierig das längste passende Wort aus einem Wörterbuch gewählt werden. Sie können davon ausgehen, dass sich der übergebene Text immer auf diese Weise in Worte aus dem Wörterbuch zerteilen lässt.

Die folgenden Definitionen werden in Beispielen verwendet:

```
val dictionary = List(
 List('F','U','N','K','T','I','O','N'),
 List('F','U','N','K','T','I','O','N','A','L','E'),
 List('P','R','O','G','R','A','M','M'),
 List('P','R','O','G','R','A','M','M','I','E','R','U','N','G'))
val example = LazyList('F','U','N','K','T','I','O','N','A','L','E',
 'P','R','O','G','R','A','M','M','I','E','R','U','N','G')
```

- a) Ergänzen Sie die Funktion `choose(ds)(cs)`, die aus der Liste von Wörtern `ds` das längste Wort auswählt und zurückgibt, das ein Präfix der möglicherweise unendlichen `LazyList cs` ist. Betrachten Sie dazu jedes Wort `d` aus `ds`. Ist `d` ein Präfix von `cs` und länger als das beste bisher gefundene Wort, soll `d` als bestes gefundenes Wort vermerkt werden. Beispiel:

```
choose(dictionary)(example)
== List('F','U','N','K','T','I','O','N','A','L','E')
```

Tipp: Signatur von `foldRight`: `foldRight[B](z: B)(op: (A, B) => B): B`.

```
def choose: List[List[Char]] => LazyList[Char] => List[Char]
= ds => cs => ds.foldRight(List[Char]())((d, r) =>
 if (
 cs.take(d.length).toList == d
 d.length < r.length
)
 r
 else d
)
```

- b) Vervollständigen Sie die Funktion `parse(ds)(cs)`, die die möglicherweise unendliche `LazyList` unter Anwendung von `choose` in eine `LazyList` von Worten zerlegt. Beispiel:

```
parse(dictionary)(example).toList == List(
 List('F','U','N','K','T','I','O','N','A','L','E'),
 List('P','R','O','G','R','A','M','M','I','E','R','U','N','G'))

def parse: List[List[Char]] => LazyList[Char] => LazyList[List[Char]]
= ds => {
```

```
 case LazyList() => LazyList()

 case cs => {

 val wort = choose(ds)(cs)
 wort #:: parse(ds)(cs.drop(wort.length))

 }
}
```

**Aufgabe 6 (Scala: Springerproblem)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Kann ein Springer so über ein Schachbrett bewegt werden, dass er jedes Feld exakt einmal besucht? In dieser Aufgabe entwickeln Sie ein Verfahren, das einen solchen Weg auf einem quadratischen Brett mit  $n \times n$  Feldern findet. Die vorgegebene Funktion `moves(n) (x, y)` ermittelt alle Möglichkeiten, einen Springer auf einem solchen Brett von Position  $(x, y)$  aus zu bewegen:

**def** moves: Int => (Int, Int) => List[(Int, Int)] = n => (x, y) => ...

Die Werte für  $x$  und  $y$  liegen dabei zwischen 0 (inklusive) und  $n$  (exklusive).

- a) Füllen Sie die Lücken in der Funktion `solved(n) (vs)`, die überprüfen soll, ob `vs` alle Felder des  $n \times n$  Felder umfassenden Bretts enthält.

```
def solved: Int => List[(Int, Int)] => Boolean = n => vs =>
 (for (x <- List(0,n)
 y <- List(0,n)
 if (!vs.contains((x,y)))
 yield (x, y)) == Nil
```

- b) Die Funktion `tour(n) (x, y)` soll für einen Springer an Position  $(x, y)$  einen Pfad über ein Schachbrett der Größe  $n \times n$  zurückgeben, falls ein solcher existiert. Andernfalls soll das Ergebnis Nil sein.

Implementieren Sie dazu die Hilfsfunktion `backtrack(vs)`, die mithilfe eines Rücksetzverfahrens (Backtracking) versucht, den Pfad `vs` zu einer Lösung zu erweitern. Das zuletzt besuchte Feld steht in `vs` dabei an erster Stelle.

- Ist `vs` eine gültige Lösung, so soll dieser Pfad zurückgegeben werden.
- Andernfalls ermittelt `backtrack` die durch einen Sprung erreichbaren unbesuchten Felder (`unvisited`). Anschließend versucht die Funktion, nacheinander den Pfad durch diese Felder zu erweitern, solange noch keine Lösung (Nil) gefunden wurde.

```
def tour: Int => (Int, Int) => List[(Int, Int)] = n => (x, y) => {
 def backtrack: List[(Int, Int)] => List[(Int, Int)] = {
 case vs if (solved(n)(vs)) => vs
 case vs => {
 val (x, y) = vs.head
 val unvisited = moves(n)(x, y).filter(
 z => !vs.contains(z)
)
 unvisited.foldLeft(Nil)({
 case (Nil, field) => backtrack(field :: vs)
 case (res, _) => res
 })
 }
 }
 backtrack(List((x, y)))
}
```

**Aufgabe 4 (Scala: Sekanten-Verfahren)**

(9 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Das Sekanten-Verfahren sucht eine Nullstelle einer Funktion. Gegeben sind folgende Definitionen:

```
// Beispielfunktion:
```

```
val f0: Double => Double = x => x*x - x - 2
```

```
// Prüft, ob zwei Zahlen annähernd gleich sind
```

```
def close: (Double, Double) => Boolean = ...
```

- a) Für eine Funktion  $f$  und zwei Startwerte  $x_0, x_1$  berechnet das Sekanten-Verfahren die Folge

$$x_n = \frac{x_{n-2} \cdot f(x_{n-1}) - x_{n-1} \cdot f(x_{n-2})}{f(x_{n-1}) - f(x_{n-2})}$$

Implementieren Sie die Funktion `secant (f) (x0, x1)` (**inklusive** Signatur). Diese liefert die **unendliche LazyList** aller Folgenglieder  $x_i$  beginnend bei  $x_0$  für die **gegebene Funktion** zurück. Ihre allgemeingültige Implementierung muss zu folgendem Beispiel passen:

```
val s: LazyList[Double] = secant(f0)(0.0, 3.0)
```

```
s.take(3).toList == List(0.0, 3.0, 1.0) // 1.0 = (0*4-3*(-2))/(4-(-2))
```

```
def secant: (Double => Double) => (Double, Double) => LazyList[Double]
```

```
= f => (x0, x1) => {
 val xn = (x0 * f(x1) - x1*f(x0)) / (f(x1) - f(x0))
 x0 #:: secant(f)(x1, xn)
}
```

- b) Möglicherweise findet das Sekanten-Verfahren **keine Nullstelle**. Dies kann durch den Scala-Datentyp `Option` ausgedrückt werden:

```
abstract class Option[+T]
```

```
case object None extends Option[Nothing]
```

```
case class Some[+T](v: T) extends Option[T]
```

Ergänzen Sie die Funktion `root (xs, n)`. Befinden sich **unter den ersten n Folgengliedern** der **unendlichen LazyList** `xs` zwei aufeinanderfolgende Zahlen, die annähernd gleich (`close`) sind, soll die **erste solche Zahl in xs** passend als `Option` verpackt zurückgegeben werden. Andernfalls liefert `root None`.

```
def root: (LazyList[Double], Int) => Option[Double] = (xs, n) => {
```

```
// fs: Alle Tupel in xs aufeinanderfolgender annähernd gleicher Zahlen
```

```
val fs = xs .zip(xs.tail).filter(p => close(p._1, p._2))
```

```
fs.toList match {
```

```
case Nil => None
```

```
case h::_ => Some(h._1)
```

```
}
```

```
}
```

**Aufgabe 6 (Scala: Picknick)**

(11 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Nach der Pandemie möchte Alice mit ihrem Freundeskreis ein Picknick veranstalten. Dazu hat sie eine **Liste der Speisen** erstellt, die die **eingeladenen Personen mögen**, z. B.:

```
type Food = String
val liked = List(List("Brot", "Paprika"), List("Wurst", "Brot"),
 List("Paprika", "Apfel"))
```

Gesucht ist nun die **günstigste Auswahl an Speisen** für **beliebige Eingabelisten**, sodass jede Person eine **gemochte Speise essen kann**. Die Kosten liefert eine Kostenfunktion, z. B.:

```
def cost: Food => Int = {
 case "Apfel" | "Wurst" => 3
 case default => 2
}
```

- a) Ergänzen Sie die Funktion `removeAll(e, ls)`, die aus **einer Liste von Listen von Speisen** `ls` diejenigen **Listen entfernen soll**, die die **Speise e enthalten**. Beispiel:

```
removeAll("Brot", liked) == List(List("Paprika", "Apfel"))
```

```
def removeAll: (Food, List[List[Food]]) => List[List[Food]]
```

```
 = (e, ls) => ls. ()
```

- b) Vervollständigen Sie die Funktion `minResult(cf, ls)`, **die aus einer Liste von Listen von Speisen `ls` diejenige zurückgibt**, deren **Summe der durch die Kostenfunktion `cf` gegebenen Kosten am niedrigsten** ist, oder **`Nil`, falls `ls` leer** ist. Beispiel:

```
minResult(cost, liked) == List("Brot", "Paprika"))
```

```
def minResult: (Food => Int, List[List[Food]]) => List[Food]
```

```
 = (cf, ls) => .foldLeft() (
 (r, e) => {
```

```
 val r_kosten = (for (s <- r) yield cf(s)).sum
 val e_kosten = (for (s <- e) yield cf(s)).sum
 if (r == Nil || e_kosten < r_kosten) e else r
```

```
 })
```

- c) Die Funktion `best(cf, ls)` sucht die günstigste Auswahl von Speisen gemäß Kostenfunktion `cf`, sodass das Ergebnis mindestens eine Speise aus jeder Liste aus `ls` enthält.

Beispiel:

```
best(cost, liked) == List("Paprika", "Brot")
```

Das heißt, dass mit den Speisen "Paprika" und "Brot" alle Freunde zufrieden sind.

Implementieren Sie dazu die Hilfsfunktion `helper(ps)(fs)`, die eine Teillösung `ps` sowie die Liste der noch unerfüllten Speisewünsche `fs` erhält, rekursiv alle Möglichkeiten betrachtet und mit `minResult` die günstigste auswählt.

Nehmen Sie an, dass `fs` keine leeren Listen und keine Listen mit in `ps` vorhandenen Speisen enthält. Garantieren Sie letzteres bei Ihren rekursiven Aufrufen mittels `removeAll`.

```
def best: (Food => Int, List[List[Food]]) => List[Food] = (cf, fs) => {

 def helper: (List[Food], List[List[Food]]) => List[Food]
 = (ps, fs) => fs match {
 // keine Vorlieben -> leeres Ergebnis
 case Nil => Nil

 // letzte Liste h: h sind unberücksichtigte Vorlieben der letzten
 // Person -> nehme Speise in Ergebnis auf, die die Kosten minimiert
 case h::Nil => minResult(cf, h.map(s => s::helper(s::ps, Nil))

 // wähle nacheinander Speisen (Food) aus h und führe Rekursion durch
 case h::ts => minResult(cf,

 h.map(s => s::helper(s::ps, removeAll(s, ts)))

)
 }

 helper(Nil, fs)
}
```

**Aufgabe 5 (Scala: Klausurstoff)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Jedes Semester aufs Neue die schwerste PFP-Klausur aller Zeiten zu stellen, ist nicht einfach. Der Trick besteht darin, immer die Themen abzuprüfen, die am wenigsten in den Übungen behandelt wurden. Ziel dieser Aufgabe ist es, diese Auswahl zu automatisieren. Die Themen einer Übung werden als `List` von `String` dargestellt, die Themen eines Semesters als Liste dieser Listen:

```
val topics = List(
 List("Java", "Petri", "Race_Condition"),
 List("Java", "Multicore", "Synchronized"),
 List("Synchronized", "Deadlock", "Java")).
```

- a) Ergänzen Sie die Funktion `flatten` so, dass sie eine `List[List[String]]` in eine `List[String]` überführt, indem sie die übergebenen Listen unter Wahrung ihrer Reihenfolge und der ihrer Elemente konkateniert:

```
flatten(List(List("A", "B"), List("C", "A"))) == List("A", "B", "C", "A")
```

```
def flatten: = exs =>
```

```
 for(
```

- b) `count` soll nun eine `List[String]` entgegennehmen, alphabetisch sortieren, Duplikate erkennen und eine Liste von Paaren zurückliefern. Jedes Paar (`word`, `n`) in der Ausgabe besagt dabei, wie oft das Wort `word` in der ursprünglichen Liste vorkam. Nutzen Sie dafür die Hilfsfunktion `helper`, die das Problem mittels Pattern-Matching lösen soll.

```
count(List("A", "B", "C", "A")) == List(("A",2), ("B",1), ("C",1))
```

```
def count: = ws => {
```

```
 def helper: List[String] => List[(String, Int)] = {
```

```
 case
```

```
 }
```

```
 helper(ws
```

```
}
```

- c) Ergänzen Sie die Funktion `getExamTopics`, die eine `List[List[String]]` `exs` und eine Anzahl `n` entgegennimmt und daraus die ersten `n` Zeichenketten in einer `List[String]` zurückgibt, die am seltensten in `exs` vorkommen. Verwenden Sie die beiden vorherigen Funktionen.

```
getExamTopics(topics) (3) == List("Deadlock", "Multicore", "Petri")
```

```
def getExamTopics: =
```

$$\text{exs} \Rightarrow n \Rightarrow \{$$

```
.sortBy(
```

}

**Aufgabe 6 (Scala: Queens)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Gegeben seien folgende Typen und Funktionen:

```

type Col = Char // chess board column index
type Row = Int // chess board row index
val cols = List('a', 'b', 'c', 'd', 'e', 'f', 'g', 'h')
val rows = List(1, 2, 3, 4, 5, 6, 7, 8)
type Field = (Col, Row) // field within chess board
def sameColumn: (Field, Field) => Boolean = (a, b) => a._1 == b._1
def sameRow: (Field, Field) => Boolean = (a, b) => a._2 == b._2
def sameDiagonal: (Field, Field) => Boolean =
 (a, b) => (a._1 - b._1).abs == (a._2 - b._2).abs
def beats: Field => Field => Boolean =
 a => b => sameColumn(a, b) || sameRow(a, b) || sameDiagonal(a, b)

```

Das Damenproblem besteht darin, acht Damen auf einem normalen Schachbrett (8×8) so zu platzieren, dass keine Dame die andere schlägt (beats).

- a) Ergänzen Sie die Funktion `getFreeRows`: Sie bekommt die Liste der von Damen besetzten Felder und soll die Liste der Damen-freien Zeilen zurückgeben.

```

def getFreeRows: List[Field] => List[Row] = qs =>
 rows.

```

- b) Ergänzen Sie folgende Funktion: Sie bekommt die Liste der von Damen besetzten Felder sowie eine noch freie Zeile und soll alle Felder dieser Zeile zurückgeben, die noch nicht von Damen *geschlagen* werden.

```

def getFreeFieldsInRow: List[Field] => Row => List[Field] = qs => row =>

```

```

 for (

```

- c) Ergänzen Sie die Funktion `solve`: Sie bekommt die Liste der bereits von Damen belegten Felder und soll sie zu einer beliebigen Lösung des Damenproblems ergänzen. Sie dürfen hier annehmen, dass es garantiert mindestens eine Lösung gibt.

```

def solve: List[Field] => List[Field] = queens => {

```

```

 match {

```

```

 case

```

```

 =>

```

```

 case

```

```

 =>

```

```

 }.foldLeft(

```

```

 }

```

```

}

```

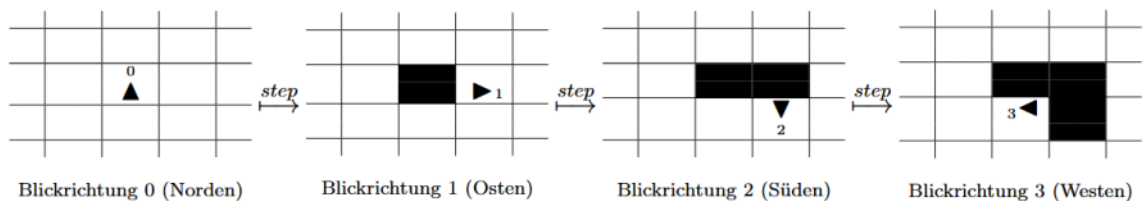
**Aufgabe 5 (Scala: Ameise)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Langtons Ameise lebt auf einem unendlich großen Raster aus anfangs weißen, viereckigen Feldern. In jedem Zeitschritt tut sie Folgendes in genau dieser Reihenfolge:

1. Steht sie auf einem weißen Feld, dreht sie sich um 90° im Uhrzeigersinn, ansonsten gegen den Uhrzeigersinn.
2. Sie ändert die Farbe ihres aktuellen Feldes: Weiße Felder werden schwarz und schwarze weiß.
3. Sie läuft auf das nächste Feld in Blickrichtung.



Ziel dieser Aufgabe ist es, Ameise und Raster zu simulieren. Dazu seien die folgenden Definitionen vorgegeben:

```
case class Ant(x : Int, y : Int, dir : Int)
// dir 0/1/2/3 entspricht Norden/Osten/Süden/Westen
case class Tile (x : Int, y : Int, black : Boolean) // Ein Feld
type Trail = List[Tile] // Genau alle besuchten Felder; jedes Feld nur einmal
type State = (Ant, Trail) // Der Gesamtzustand

// Aenderung von x und y pro Schritt fuer jede der vier Blickrichtungen
val stepDelta = List((0,1), (1,0), (0,-1), (-1, 0))

// Liefert die Blickrichtung, die die Ameise als naechstes annimmt
def newDir : State => Int = // ...

// Liefert die Farbe an den gegebenen Koordinaten
def getColor : ((Int, Int), State) => Boolean = // ...
```

- a) Ergänzen Sie die Funktion `updateColor`, die das `Tile` an der Stelle `(x,y)` in der Farbe `b` einfärbt. Achten Sie darauf, dass sich jede Koordinate nur einmal in einem `Trail` befinden darf.

```
def updateColor : ((Int, Int), Boolean, State) => State = {
 case ((x, y), b, (ant, trl)) =>
 () }
```

- b) Ergänzen Sie nun die Funktion `updateAnt`, die die Ameise auf das gegebene Feld  $(x, y)$  mit Richtung `dir` setzt. Sollte das Feld noch nicht existieren, muss es erstellt werden.

```
def updateAnt : ((Int, Int, Int), State) => State = {
 case ((x, y, dir), (ant, trl)) =>
 (,
 if()
 else) }
```

- c) Vervollständigen Sie die Funktion `step`. Sie erhält einen Zustand `s` und gibt den Folgezustand nach einem Zeitschritt zurück.

```
def step : State => State = s => {
 val (ant, trl) = s; val nd = newDir(s);
```

```
}
```

- d) Schreiben Sie schließlich die Funktion `simulation`, welche für einen gegebenen Startzustand `s` eine unendliche Liste aus `s` sowie allen seinen Folgezuständen zurückliefert.

```
def simulation : State => LazyList[State] = s =>
```

**Aufgabe 6 (Scala: Unäre Operationen)**

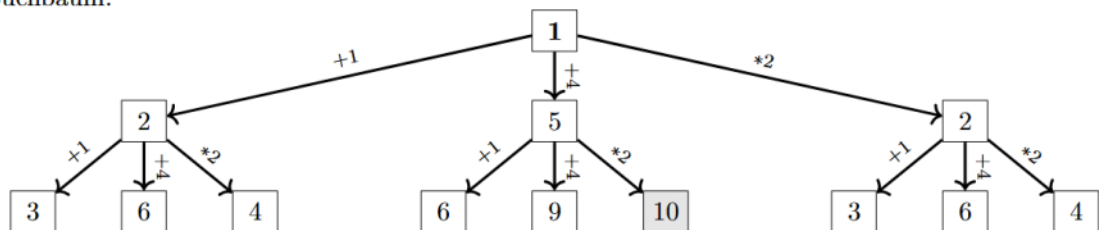
(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Gegeben seien ein Start- und Endwert (`Int`) sowie eine Menge an Funktionen (`Func`). Ziel dieser Aufgabe ist es, eine minimale Folge von Anwendungen dieser Funktionen zu finden, die den Start- in den Endwert überführt (ohne Berücksichtigung des Operatorvorrangs). Diese Sequenz soll in der umgekehrten Reihenfolge der Funktionsanwendungen zurückgeliefert werden. Verwenden Sie hierfür Breitensuche.

```
type Func = (Int => Int)
type Level = List[(Int, List[Func])] // (Wert, Funktionsfolge zu Wert)
val functionList : List[Func] = List(_+1, _+4, _*2)
```

Für die gegebene `functionList`, den Startwert **1** und den Endwert **10** ergibt sich folgender Suchbaum.



| Level 1                     | Level 2                                                           | Level 3                                                                               |
|-----------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>List((1, Nil))</code> | <code>List((2, List(_+1)), (5, List(_+4)), (2, List(_*2)))</code> | <code>List((3, List(_+1, _+1)), (6, List(_+4, _+1)), (4, List(_*2, _+1)), ...)</code> |

- a) Vervollständigen Sie die Funktion `newLevel`, die aus der Liste der verfügbaren Funktionen und einer Ebene des Baumes die nächste Ebene berechnet.

```
newLevel(functionList)(List((1, Nil))) =
 List((2, List(_+1)), (5, List(_+4)), (2, List(_*2)))
```

```
def newLevel: List[Func] => Level => Level = funcs => lvl =>
```

```
 for (
```

```
)
```

```

```

- b) Vervollständigen Sie nun die Funktion `findValue`. Sie erhält einen Zielwert und eine Ebene und liefert eine passende Funktionsfolge zu diesem Wert zurück. Sollte der Wert nicht in der Ebene enthalten sein, soll `None` zurückgegeben werden.

```
findValue(42)(List((22, List(_+1, _+1)), (42, List(_+1, _*2)))) .get =
 List(_+1, _*2)
```

```
def findValue : Int => Level => Option[List[Func]] = value => lvl =>
```

```
 lvl.foldLeft[Option[List[Func]]]() (
```

```
)
```

- c) Ergänzen Sie schließlich die Funktion `findShortest`, die mit der ihr gegebenen Liste an Funktionen den kürzesten Weg vom Start- zum Zielwert ermittelt und die entsprechende Folge an Funktionen zurückliefert.

```
findShortest(functionList)(1,18) = List(_*2,_+4,_+4) // (1+4+4)*2 = 18
```

```
def findShortest: List[Func] => (Int, Int) => List[Func] =
 funcs => (start, end) => {
 def helper: Level => List[Func] = lvl => {
 match {
 case None =>
 case
 }
 }

 }
}
```

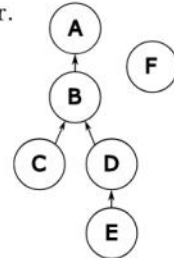
**Aufgabe 5 (Scala: LCA)**

(8 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

In dieser Aufgabe soll das Lowest-Common-Ancestor-Problem für Bäume gelöst werden. Jeder Knoten (Node) enthält dabei einen garantiert einzigartigen Namen und, solange es sich um keinen Wurzelknoten handelt, eine Referenz auf seinen direkten Vorgänger.

```
case class Node(name : String, pred : Option[Node])
val a = Node("A", None)
val b = Node("B", Some(a))
val c = Node("C", Some(b))
val d = Node("D", Some(b))
val e = Node("E", Some(d))
val f = Node("F", None)
```



- a) Vervollständigen Sie die Funktion `ancestors`, so dass ihre Signatur zu den Beispielen passt. Sie soll für einen gegebenen Knoten die Liste der Namen aller seiner Vorgänger sowie des Knotens selbst zurückliefern. Achten Sie darauf, dass Sie dabei die Reihenfolge von der Wurzel zum Knoten einhalten. Sie dürfen davon ausgehen, dass die Struktur frei von Zyklen ist.

```
ancestors(d) = List("A", "B", "D")
ancestors(f) = List("F")
```

```
def ancestors : {
 case
 case
}
```

- b) Vervollständigen Sie nun die Funktion `lastMatch`. Sie erhält zwei Listen von Strings. Stimmt der Anfang beider Listen überein, soll eine Option mit dem **letzten** übereinstimmenden String zurückgegeben werden, ansonsten `None`.

```
lastMatch(List("A", "B", "C"), List("A", "B", "D")) = Some("B")
```

```
def lastMatch : {
 case
}
```

- c) Ergänzen Sie schließlich die Funktion `lca`, die den Namen des letzten gemeinsamen Vorgängers `p` zweier Knoten als `Some(p)` zurückliefert, oder `None`, falls diese Knoten nicht zum selben Baum gehören.

```
lca(a,b) = Some("A") lca(e,c) = Some("B") lca(e,f) = None
```

```
def lca :

```

**Aufgabe 6 (Scala: Sudoku)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Das Ziel beim Sudoku ist es, ein teilweise ausgefülltes Raster mit den Ziffern von 1 bis 9 zu vervollständigen. Das Raster hat immer die Abmessungen  $9 \times 9$  und setzt sich wiederum aus neun  $3 \times 3$  Quadraten zusammen. Jede Ziffer darf in jeder Zeile, jeder Spalte sowie jedem Quadrat nur einmal vorkommen.

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 |   | 9 |   |   | 8 |   |   |   |
|   |   |   |   | 7 |   |   | 8 | 3 |
|   |   |   |   | 2 |   |   |   | 7 |
| 2 |   |   | 8 |   | 3 | 1 |   |   |
| 3 | 6 | 1 |   |   |   | 2 |   | 8 |
|   |   |   | 4 |   |   | 6 |   |   |
| 9 |   |   | 1 | 8 | 6 |   | 5 |   |
|   | 1 | 8 |   |   | 4 |   | 9 | 6 |
|   | 5 | 4 |   |   | 7 |   | 2 |   |

→

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 7 | 9 | 3 | 4 | 8 | 5 | 6 | 2 |
| 4 | 2 | 6 | 5 | 7 | 1 | 9 | 8 | 3 |
| 5 | 8 | 3 | 6 | 2 | 9 | 4 | 1 | 7 |
| 2 | 4 | 5 | 8 | 6 | 3 | 1 | 7 | 9 |
| 3 | 6 | 1 | 7 | 9 | 5 | 2 | 4 | 8 |
| 8 | 9 | 7 | 4 | 1 | 2 | 6 | 3 | 5 |
| 9 | 3 | 2 | 1 | 8 | 6 | 7 | 5 | 4 |
| 7 | 1 | 8 | 2 | 5 | 4 | 3 | 9 | 6 |
| 6 | 5 | 4 | 9 | 3 | 7 | 8 | 2 | 1 |

Das Problem soll mit Hilfe eines Rücksetzverfahrens gelöst werden. Gehen Sie im Folgenden davon aus, dass die Vorgabe die oben beschriebenen Regeln erfüllt und nicht ausgefüllte Zellen mit der Ziffer 0 dargestellt werden.

```

val nums = List.range(1,10)
type Grid = List[List[Int]]

// Ersetzt den Wert in Zelle (row,column) mit value
def updatedCell : (Grid,Int,Int,Int) => Grid = (grid,row,column,value) => ...

// Erstellt eine Liste der bereits verwendeten Werte in Zeile row
def numbersInRow : (Grid,Int) => List[Int] = (grid,row) => ...

// Erstellt eine Liste der bereits verwendeten Werte in Spalte column
def numbersInColumn : (Grid,Int) => List[Int] = (grid,column) => ...

```

- a) Ergänzen Sie die Funktion `numbersInSquare`, die für eine gegebene Position (`row`, `column`) die Liste aller Ziffern liefert, die sich im selben  $3 \times 3$  Quadrat befinden. Diese Liste darf 0 enthalten.

```

def numbersInSquare : (Grid,Int,Int) => List[Int] =
 (grid,row,column) =>
 val scr = row/3*3 // square corner row
 val scc = column/3*3 // square corner column
 for ()
 yield ()

```

- b) Ergänzen Sie nun die Funktion `optionsForCell`. Sie liefert für eine gegebene Position `(row,column)` eine Liste aller Ziffern zurück, die dort eingesetzt werden können (also in der selben Zeile, der selben Spalte sowie im selben Quadrat noch nicht vergeben sind).

```
def optionsForCell : (Grid,Int,Int) => List[Int] = (grid,row,column) =>
```

```
 val nR = numbersInRow()
```

```
 val nC =
```

```
 val nS =
```

```
 nums.
```

- c) Die Funktion `solve` soll nun für ein teilweise ausgefülltes Raster die Liste aller gültigen Lösungen zurückliefern. Hierfür generiert die Hilfsfunktion `helper` für eine gegebene Position die Liste aller möglichen Werte, setzt jeden davon ein und versucht rekursiv für jeden davon eine Lösung zu konstruieren. Achten Sie darauf, dass Sie die bereits vorgegebenen Ziffern einhalten. Existiert keine gültige Lösung, soll `Nil` zurückgegeben werden.

```
def solve : Grid => List[Grid] = grid =>
```

```
 def helper : (Grid,Int,Int) => List[Grid] = {
```

```
 // Lösung gefunden (letzte Zeile fertig):
```

```
 case
```

```
 // Zeile fertig:
```

```
 case
```

```
 // Zelle vorgegeben:
```

```
 case
```

```
 case (grid,row,column) =>
```

```
 val options = optionsForCell(grid,row,column)
```

```
 options.foldLeft[List[Grid]]()
```

```
 (=>
```

```
 helper())
```

```
 }
```

```
 helper()
```

**Aufgabe 5 (Scala: Petri-Netz)**

(10 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

In dieser Aufgabe sollen Petri-Netze ohne Stellenkapazitäten in Scala simuliert werden. Jede Stelle (Place) wird als Tupel aus ihrem Namen (`id`) und ihren aktuell gehaltenen Token dargestellt, jede Kante (Edge) als Tupel aus dem Namen der verbundenen Stelle und ihrer Gewichtung. Eine negative Gewichtung signalisiert dabei eine ausgehende Kante. Eine Transition ist ein Tupel aus ihrem Namen und der Liste der verbundenen Kanten.

```
type Place = (String, Int)
type Edge = (String, Int)
type Transition = (String, List[Edge])
type PetriNet = (List[Place], List[Transition])
```

```
// Example
```

```
val a = ("A", 1)
```

```
val b = ("B", 0)
```

```
val c = ("C", 0)
```

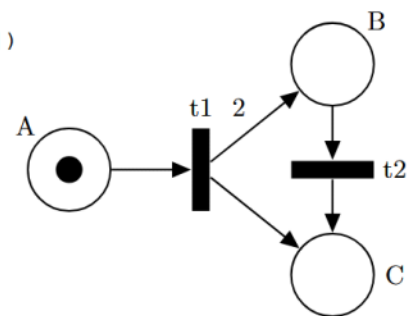
```
val places = List(a, b, c)
```

```
val t1 = ("t1", List(("A", -1), ("B", 2), ("C", 1)))
```

```
val t2 = ("t2", List(("B", -1), ("C", 1)))
```

```
val transitions = List(t1, t2)
```

```
val net = (places, transitions)
```



- a) Vervollständigen Sie die Funktion `getPlaceById`. Sie erhält den Bezeichner einer Stelle sowie ein Petri-Netz und gibt die entsprechende Stelle im Netz zurück.

```
getPlaceById("B")(net) = ("B", 0)
```

```
def getPlaceById: String => PetriNet => Place = idP =>
```

```
 case (ps, _) => ps.filter
```

- b) Vervollständigen Sie die Funktion `updatedPlace`. Sie erhält einen Bezeichner (`idP`), eine Tokenanzahl (`deltaTs`) und eine Stelle. Sind der gegebene Bezeichner und der Name der gegebenen Stelle identisch, wird `deltaTs` zu deren Tokenanzahl addiert und das resultierende neue Tupel zurückgegeben. Ansonsten wird die gegebene Stelle unverändert zurückgegeben.

```
updatedPlace("B", 2)(a) = ("A", 1)
```

```
updatedPlace("B", 2)(b) = ("B", 2)
```

```
def updatedPlace: (String, Int) => Place => Place = (idP, deltaTs) =>
```

```
 case (id, ts) if
```

```
 case
```

- c) Vervollständigen Sie nun die Funktion `updatedPN`, die `updatedPlace` nutzt, um `deltaTs` zur Tokenanzahl des Platzes mit dem Bezeichner `idP` im Petri-Netz zu addieren.

```
updatedPN("B",2)(net) = (List(("A",1), ("B",2), ("C",0)),List(...))
```

```
def updatedPN: (String, Int) => PetriNet => PetriNet = (idP, deltaTs) =>
 case (ps,ts) =>
```

- d) Vervollständigen Sie anschließend die Funktion `isEnabled`. Sie erhält eine Transition und ein Petri-Netz und überprüft, ob die Transition aktuell schalten darf.

```
isEnabled(t1)(net) = true
```

```
isEnabled(t2)(net) = false
```

```
def isEnabled: Transition => PetriNet => Boolean = t => pn =>
```

```
 t._2.forall
```

- e) Vervollständigen Sie schließlich die Funktion `fire`. Sie erhält eine Transition und ein Petri-Netz. Kann die Transition im gegebenen Netz nicht schalten, wird `None` zurückgegeben. Ansonsten wird das resultierende Netz nach dem Schalten von `t` zurückgegeben.

```
fire(t1)(net) = Some((List((A,0), (B,2), (C,1)),List(...)))
```

```
fire(t2)(net) = None
```

```
def fire: Transition => PetriNet => Option[PetriNet] = t => pn =>
```

```
 if
```

```
 Some(t._2.foldLeft
```

```
 else
```

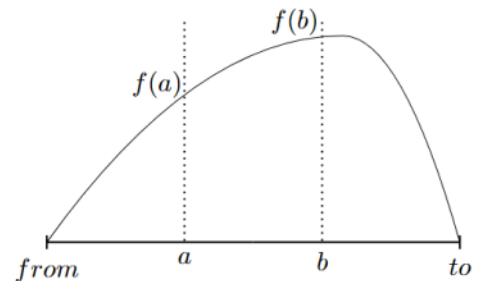
```
 None
```

**Aufgabe 6 (Scala: Ternäre Suche)**

(8 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Eine unimodale Funktion ist eine Funktion mit einem eindeutigen (lokalen und globalen) Maximum. Dieses Maximum einer unimodalen Funktion  $f$  kann mit der ternären Suche gefunden werden. Dafür wird ein Suchintervall  $(from, to)$  anhand von zwei Stellen  $a, b$  ( $a < b$ ) in drei gleich große Abschnitte unterteilt. Ist anschließend  $f(a) < f(b)$  kann der linke der drei Abschnitte verworfen werden, ansonsten der rechte. Das resultierende kleinere Intervall wird rekursiv weiter durchsucht.



```
type Function = Double => Double
val eps = 0.001
```

- a) Vervollständigen Sie die Funktion `divide`. Sie teilt das gegebene Intervall  $(from, to)$  in drei gleich große Abschnitte und liefert die beiden Stellen  $a$  und  $b$  als Tupel zurück.

```
def divide: (Double, Double) => (Double, Double) = (from, to) =>
```

- b) Vervollständigen Sie nun die Funktion `step`. Sie führt einen Schritt der ternären Suche für die Funktion  $f$  auf dem Intervall  $(from, to)$  aus und liefert die neuen Intervallgrenzen als Tupel zurück.

```
def step: Function => (Double, Double) => (Double, Double) =
 f => (from, to) =>
```

|             |              |
|-------------|--------------|
|             | <b>match</b> |
| <b>case</b> |              |
| <b>case</b> |              |

- c) Ergänzen Sie anschließend die Funktion `sequence`, die für eine Funktion und ein Startintervall die unendliche Liste aus diesem Intervall und allen Folgeintervallen, die sich aus rekursiver Anwendung von `step` auf das jeweils zuletzt erzeugte Intervall ergeben, zurückliefert.

```
def sequence: Function => ((Double, Double))
=> LazyList[(Double, Double)] = f =>
 case (from, to) =>
```

- d) Vervollständigen Sie die Funktion `findMaximum`, die mit Hilfe von `sequence` das Maximum einer unimodalen Funktion  $f$  im Intervall  $(-100, 100)$  bestimmt. Das Maximum gilt als gefunden, sobald das Suchintervall nicht mehr größer als `eps` ist.

```
def findMaximum: Function => Double = f =>
```

```
sequence() ()
```

**Aufgabe 5 (Scala: Monte-Carlo)**

(7 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind *var* und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Wenn  $a$  und  $b$  gleichverteilte Zufallsvariablen zwischen 0 und 1 sind, liegt der Punkt  $(a, b)$  mit einer Wahrscheinlichkeit von  $\frac{\pi}{4}$  im Einheitskreis (Kreis um den Ursprung mit Radius 1). In dieser Aufgabe soll aus einer unendlichen LazyList solcher Zahlen mithilfe einer Monte-Carlo-Simulation die Kreiszahl  $\pi$  berechnet werden.

```
// Unendlich viele Zufallszahlen zwischen 0 und 1
```

```
val valuesList : LazyList[Double] = ...
```

- a) Vervollständigen Sie die Funktion `pairing`. Sie erhält eine *unendliche* Liste von Werten und bildet aus jeweils zwei aufeinander folgenden Werten Paare, die wiederum als *unendliche* Liste zurückgegeben werden.

```
pairing(LazyList(1, 2, 3, 4, ...)) = LazyList((1,2), (3,4), ...)
```

```
def pairing: LazyList[Double] => LazyList[(Double,Double)] =
```

- b) Vervollständigen Sie nun die Funktion `testN`. Sie soll die ersten  $n$  Zahlenpaare der übergebenen LazyList parallel dahingehend überprüfen, ob diese im Einheitskreis liegen. Das Ergebnis dieser Berechnung soll als List der Länge  $n$  zurückgegeben werden.

```
testN(LazyList((0.1,0.2), (0.8,0.9), ...))(2) = List(true, false)
```

```
def testN: LazyList[(Double,Double)] => Int => List[Boolean] =
 pairs => n =>
```

- c) Vervollständigen Sie die Funktion `pi`. Sie wendet mithilfe der Werte `valuesList` eine Monte-Carlo-Simulation mit  $n$  Punkten an, um  $\pi$  zu berechnen und zurückzugeben.

```
pi(10000000) = 3.1416
```

```
def pi: Int => Double = n =>
```

**Aufgabe 6 (Scala: Kruskal)**

(11 Punkte)

Verwenden Sie ausschließlich aus der Lehrveranstaltung bekannte Scala-Konstrukte, dabei sind `var` und seiteneffektbehaftete Methoden/Klassen/Objekte strikt untersagt.

Der Algorithmus von Kruskal berechnet den minimalen Spannbaum eines zusammenhängenden, ungerichteten, gewichteten Graphen. Hierzu werden die Kanten des Graphen von der billigsten bis zur teuersten betrachtet und zur Lösung hinzugefügt, falls ihre beiden Enden bisher noch nicht (direkt oder indirekt) verbunden waren.

```
// Knoten haben eindeutigen Namen (String)
case class Edge(a: String, b: String, cost: Int)
// Graph als Kantenliste
type Graph = List[Edge]
```

- a) Vervollständigen Sie die Funktion `withoutNode`. Sie erhält einen Graphen `g` und einen Knoten `i` und gibt denjenigen Graphen zurück, der aus `g` dadurch hervorgeht, dass alle Kanten weggelassen werden, die im Knoten `i` beginnen oder enden.

```
def withoutNode: Graph => String => Graph = g => i =>
 g. (e => e match
```

```
)
```

- b) Vervollständigen Sie nun die Funktion `directNeighbors`. Sie liefert für einen Graphen `g` und einen Knoten `i` alle Knoten zurück, die in `g` über genau eine Kante mit `i` verbunden sind.

```
def directNeighbors: Graph => String => List[String] = g => i =>
 for ()

```

- c) Vervollständigen Sie die Funktion `connected`. Sie stellt *rekursiv* fest, ob zwei Knoten `a` und `b` im Graphen `g` direkt oder indirekt verbunden sind. Ein Knoten `a` ist mit `b` verbunden, wenn `a` und `b` identisch sind, oder mindestens einer seiner direkten Nachbarn mit `b` verbunden ist.

*Hinweis:* Knoten `a` muss für die Rekursion aus `g` entfernt werden, um eine Endlosrekursion zu vermeiden.

```
def connected: Graph => (String,String) => Boolean = g => (a,b) =>
 if ()
 else
 directNeighbors(g)(a).exists(n =>
)
```

- d) Vervollständigen Sie schließlich die Funktion `kruskal`. Sie wendet den Algorithmus von Kruskal auf den zusammenhängenden Graphen `g` an, und gibt den resultierenden Spannbaum zurück.

```
def kruskal: Graph => Graph = g =>
```

```
 val sorted =
```

```
 sorted.foldLeft[Graph] (
```

```
)
```